



UNIVERSIDAD
DE GRANADA



Técnicas de los Sistemas Inteligentes

Grado en Ingeniería Informática

Curso 2025-26. Seminario 2 Razonamiento con Restricciones

Pablo Mesejo Santiago y David Criado Ramón

Departamento de Ciencias de la
Computación e Inteligencia Artificial
<http://decsai.ugr.es>

Stuart Russell and Peter Norvig, *Artificial Intelligence: A modern Approach*, 4ª Edición, Ed. Pearson, 2022. Capítulo 5.

Armin Biere, Marijn Heule, Hans van Maaren and Toby Walsh, eds. *Handbook of satisfiability*. Vol. 185. IOS press, 2009.

En la práctica anterior hemos trabajado “implícitamente” con restricciones. Por ejemplo: no pasar sobre una catapulta (salvo si tienes dinero), no pasar por el agua (salvo si vas volando), no atravesar el portal (salvo que tengas una llave), nunca atravesar muros, mantenerte alejado de un enemigo, etc.



En general, en cualquier videojuego podríamos encontrar este tipo de restricciones (sortear obstáculos, evitar enemigos, etc.).

La **satisfacción de restricciones** (o **razonamiento con restricciones**) es una de las **áreas clásicas de la IA**.

Otras áreas clásicas de IA incluirían problemas de *búsqueda*, problemas de *planificación*, problemas de razonamiento en presencia de *incertidumbre*, problemas de *aprendizaje*,...

Es común encontrar estos problemas en **numerosas aplicaciones**:

- diseño y verificación de circuitos electrónicos,
- asignación de recursos (*scheduling*),
- planificación de producción y procesos industriales,
- verificación de software,
- resolución de puzzles y juegos,
- problemas de optimización,
- gestores de paquetes (Ubuntu, Python),
- control de versiones distribuido (Git),
- etc.

Por ello, **resolverlos eficientemente resulta crucial**.

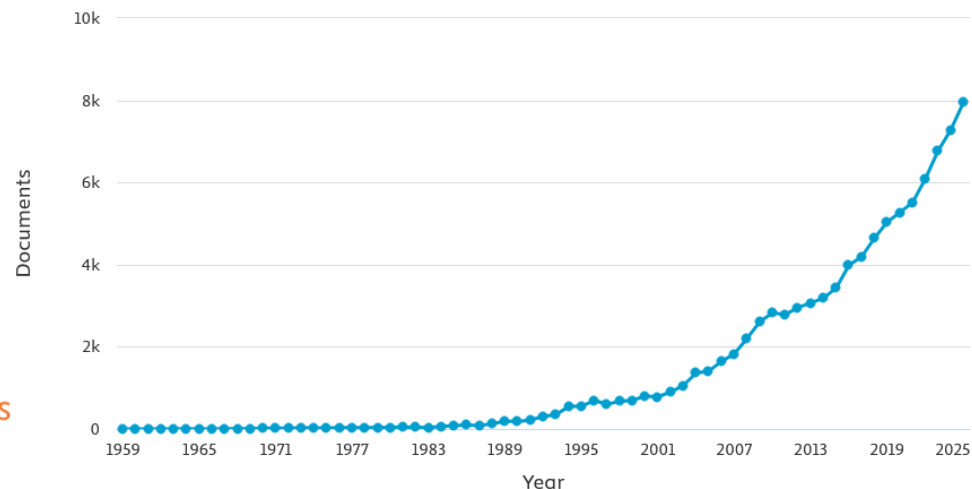
- Teorema de Cook-Levin (1971) establece que el problema de **satisfacibilidad booleana (SAT)** es **NP-completo**.
 - El problema SAT consiste en saber si, dada una expresión booleana (fórmula proposicional), hay alguna asignación de valores para sus variables que hace que la expresión sea verdadera.
 - Por ejemplo, $(x \vee y) \wedge z$ tiene como modelos (asignaciones que satisfacen la fórmula): $\{x, y, z\}, \{x, \bar{y}, z\}, \{\bar{x}, y, z\}$
- ¿Se puede resolver SAT en tiempo polinómico?
 - **P vs NP**: Uno de los [7 problemas matemáticos del milenio](#) (1M\$)
 - Problema fundamental de la computación.

95.532 document results

TITLE-ABS-KEY(((constraint AND satisfaction AND problems) OR (constraint AND optimization AND problems)) AND (LIMIT-TO (SUBJAREA, "COMP")))

"P vs NP" plantea si todos los problemas cuyas **soluciones pueden verificarse en tiempo polinómico (NP)** también **pueden resolverse en tiempo polinómico (P)**.

Documents by year



Scopus

Un **problema de satisfacción de restricciones** (del inglés, **CSP**, *constraint satisfaction problem*) se suele expresar como una **tupla** con tres elementos:

- X es un conjunto de **variables**.
- D es un conjunto de **dominios** para X.
- C es un conjunto de **restricciones** sobre X.

Ejemplo: tenemos dos números enteros positivos de forma que uno es mayor que el otro. ¿Cómo se podría expresar este problema como un CSP?

Un **problema de satisfacción de restricciones** (del inglés, **CSP**, *constraint satisfaction problem*) se suele expresar como una **tupla** con tres elementos:

- X es un conjunto de **variables**.
- D es un conjunto de **dominios** para X.
- C es un conjunto de **restricciones** sobre X.

Ejemplo: tenemos dos números enteros positivos de forma que uno es mayor que el otro. ¿Cómo se podría expresar este problema como un CSP?

Solución 1:

- $X = \{ x, y \}$
- $D = \{ \mathbb{Z}, \mathbb{Z} \}$
- $C = \{ x > 0, y > 0, x > y \}$

Solución 2:

- $X = \{ x, y \}$
- $D = \{ [1, \infty), [1, \infty) \}$
- $C = \{ x > y \}$

Dominios y restricciones son conceptos relacionados; los dominios se pueden acotar con restricciones.

Un **problema de optimización de restricciones** (del inglés, **COP**, *constraint optimization problem*) se suele expresar como una **tupla** con cuatro elementos:

- $\langle X, D, C \rangle$ como en los CSP.
- f es una **función de coste** sobre X a **minimizar/maximizar**.

Ejemplo: tenemos dos números enteros positivos de forma que uno es mayor que el otro, y la suma de ambos es **mínima**.
¿Cómo se podría expresar este problema como un COP?

Solución:

- $X = \{ x, y \}$
- $D = \{ \mathbb{Z}, \mathbb{Z} \}$
- $C = \{ x > 0, y > 0, x > y \}$
- $f = x+y$ (minimizar)

El **problema de coloreado de mapas** es otro ejemplo de problema de satisfacción de restricciones:

- **Variables:** las regiones del mapa.
- **Dominios:** los posibles colores (comunes para todas las regiones).
- **Restricciones:** dos regiones limítrofes no pueden tener el mismo color.

$$X = \{WA, NT, SA, Q, NSW, V, T\}$$

$$D = \{\text{rojo, verde, azul}\}$$

$$C1: WA \neq NT$$

$$C2: WA \neq SA$$

$$C3: NT \neq SA$$

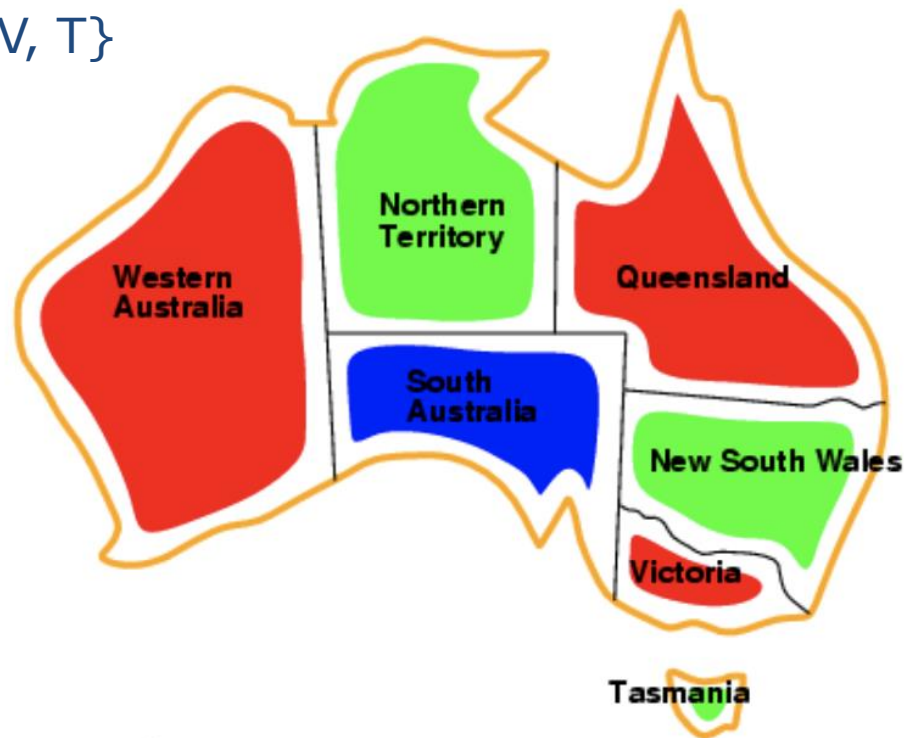
$$C4: NT \neq Q$$

$$C5: SA \neq Q$$

$$C6: SA \neq NSW$$

$$C7: SA \neq V$$

$$C8: Q \neq NSW$$

$$C9: NSW \neq V$$


Existen distintas técnicas para resolver problemas de satisfacción de restricciones:

- **Constraint Programming (CP)**: **Paradigma general** para resolver (cualquier) CSP o COP. Las técnicas de resolución se basan principalmente en *búsqueda y propagación de restricciones* y/o técnicas de *búsqueda local* (estocástica).
- **Satisfacción Booleana (SAT)**: Caso particular de CSP donde todas las variables tienen un **dominio binario** (*true/false*). SAT es el primer problema demostrado **NP-completo**.
Los SAT *solvers* modernos usan técnicas muy especializadas y, cuando un problema admite una codificación booleana compacta y natural, suelen ser muy eficientes.
- **Linear Programming (LP)**: Problemas de optimización en los que **tanto la función objetivo como las restricciones son lineales**:
$$a \cdot x_1 + b \cdot x_2 + c \cdot x_3 + \dots + k \cdot x_n \geq C$$

La región factible puede **interpretarse geométricamente**. Un algoritmo clásico para resolverlos es **SIMPLEX**.

Existen distintas técnicas para resolver problemas de satisfacción de restricciones:

- **Pseudo-Boolean Constraints (PB): Caso particular de LP donde las variables tienen dominio binario.** Están relacionadas con la programación lineal entera binaria y con SAT, y cuentan con abundante estudio teórico y herramientas de resolución especializadas.
- **SAT Modulo Theories (SMT): Generaliza SAT a fórmulas más complejas con restricciones pertenecientes a teorías formales** que involucran números reales, enteros y diversas estructuras de datos, como listas, *arrays*, vectores de bits y cadenas.
- **Answer Set Programming (ASP): Paradigma de programación declarativa** orientado especialmente a **problemas de búsqueda combinatoria y representación del conocimiento** (*knowledge representation*). Se basa en la semántica de *answer sets*, y se usa en tareas de planificación y razonamiento.

¿No basta con las técnicas de búsqueda vistas en IA (asignatura de 2º) y en la P1?

- Esas técnicas se conocen como **"atomic state-space search"**:
 - **Cada estado se trata como una caja negra** (un estado completo, indivisible), **y el algoritmo solo sabe cuál es el estado actual, qué sucesores tiene, y si un estado es meta o no.**
 - No explota la estructura interna del problema.
 - Es como decir: *"Voy generando estados y probando."*
- En un CSP no representas el problema como "estados atómicos", sino como **variables, dominios y restricciones**.
 - En lugar de "probar configuraciones completas", **haces razonamiento estructurado sobre una solución parcial.**
 - Es como decir: *"Voy construyendo una asignación y, cada vez que asigno algo, realizo deducciones sobre lo demás."*
 - Eso hace que muchos problemas intratables como búsqueda ciega, o incluso heurística, pasen a ser mucho más tratables como CSP (**"many problems that are intractable for atomic state-space search can be solved quickly when formulated as a CSP"**, Russell&Norvig 4th ed. p.165).

Hay tres grandes diferencias entre todas estas técnicas:

- **Codificación** del problema:
 - Dado un problema, **¿cómo lo represento?**
 - ¿Qué variables hacen falta? ¿Qué restricciones se necesitan?
 - Una **codificación correcta y completa** es aquella capaz de encontrar TODAS las soluciones y rechazar TODO lo que no es solución.
- Algoritmos para **resolver el problema**.
 - Dada una instancia, **¿cómo se resuelve?**
 - En general todos aplican (alguna variante de algún método de) **búsqueda**, pero existen muchos métodos (búsqueda en profundidad, búsqueda local, etc.).
- **Heurística** usada por el algoritmo de búsqueda.
 - **¿Por dónde buscar?**
 - La heurística es normalmente considerada como uno de los componentes de cada algoritmo de resolución.

Resolver CSP es **NP-completo** ...

... ¿pero eso, exactamente, qué significa?

Si tengo un problema con "n" variables, **cualquier algoritmo** requiere **en el peor caso** " $\exp(n)$ " pasos para resolverlo. Es decir, el algoritmo tiene una **complejidad $O(\exp(n))$**

3⁷ posibles asignaciones (2187) en el caso del coloreado del mapa de Australia (7 regiones y 3 colores)

Pero no siempre estamos en el peor caso

O este peor caso no siempre es "igual de malo"...

Por eso, la **codificación**, los **algoritmos de resolución** (y las **heurísticas**) usadas para resolver un determinado problema pueden **afectar considerablemente su resolución**. Veamos un ejemplo.

Pigeonhole Principle [Dirichlet's box principle]: $\text{PHP}(n,k)$

n pigeons:



si n objetos se distribuyen en k contenedores, con $n > k$, entonces al menos habrá un contenedor con más de un objeto.

k holes:

1	2	3	...	k
---	---	---	-----	---

Nuestro objetivo: asignar cada paloma a un palomar de forma que todo palomar tiene como máximo una paloma asignada.

Es fácil ver que si $n \leq k$, entonces se puede satisfacer el problema. En cambio, si $n > k$, el problema es insatisfactible.

Veamos dos **CODIFICACIONES** de $PHP(n,k)$ para cualquier $n,k > 0$

Variables (en ambas codificaciones):

- $n \cdot k$ variables Booleanas:
- $x(i,j)$ representa que la paloma "i" está en el palomar "j"

Codificación SAT:

- Toda paloma está en al menos un palomar:
 $x(i,1) \vee x(i,2) \vee \dots \vee x(i,k)$. para toda paloma "i"
- Todo palomar tiene como máximo una paloma, es decir, si en un palomar hay una paloma, no hay otra en el mismo:
 $x(i,j) \rightarrow !x(i',j)$. para toda paloma " $i \neq i'$ " y palomar "j"

Codificación PB:

- Toda paloma está en al menos un palomar:
 $x(i,1) + x(i,2) + \dots + x(i,k) \geq 1$. para toda paloma "i"
- Todo palomar tiene como máximo una paloma:
 $x(1,j) + x(2,j) + \dots + x(n,j) \leq 1$. para todo palomar "j"

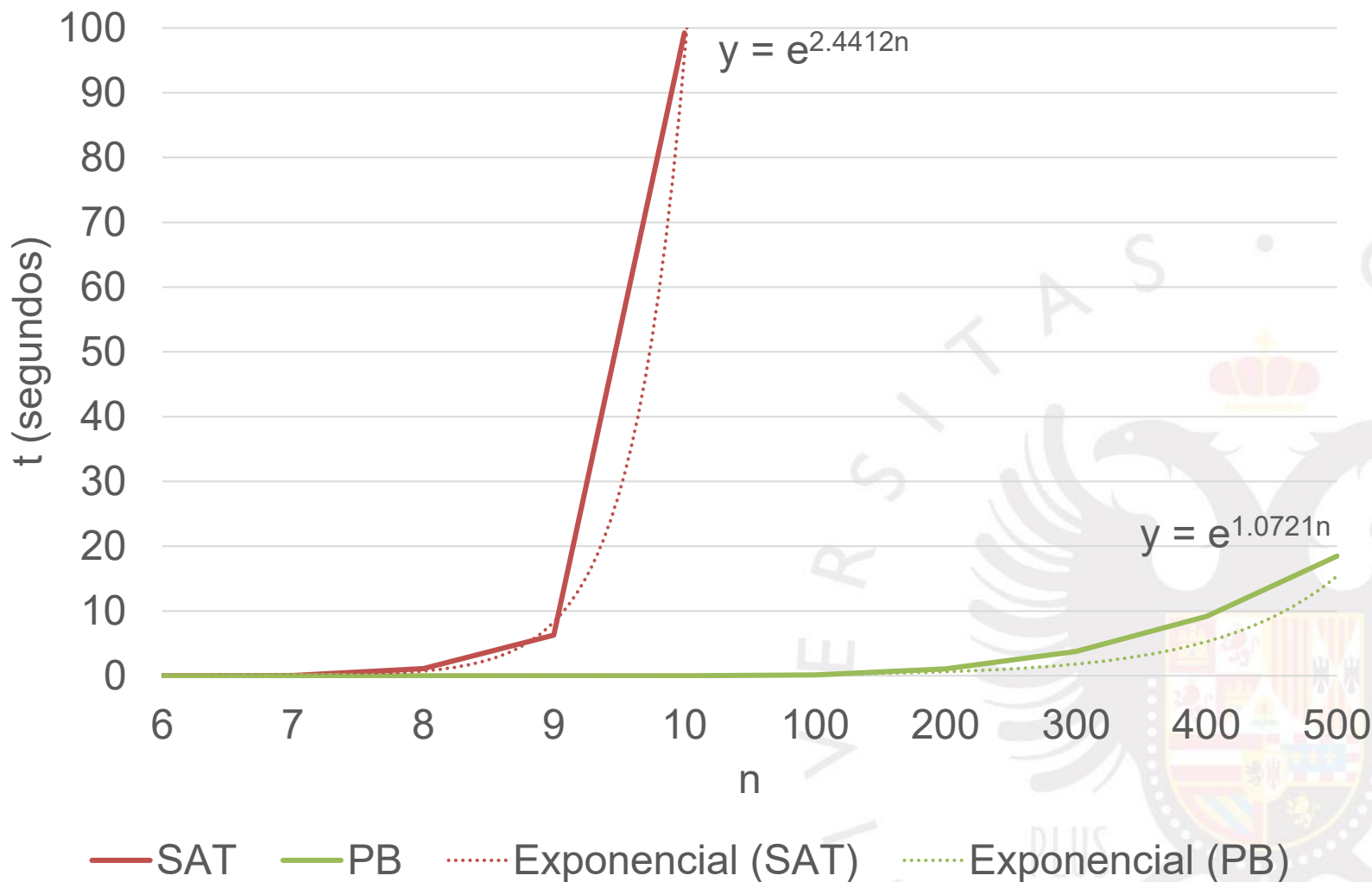
¿Cuánto se tarda en resolver estas dos fórmulas?

Palomas

Palomares

	PHP ($n+1$, n)	
n	SAT	PB
6	0.005277 s	0.004187 s
7	0.055557 s	0.005532 s
8	1.088930 s	0.006287 s
9	6.283010 s	0.007432 s
10	99.242400 s	0.008916 s
...		
100	?	0.128000 s
200	?	1.052000 s
300	?	3.760000 s
400	?	9.160000 s
500	?	18.468000 s

Comparación codificaciones PHP (n+1,n)



¿Y **cómo elegir** la codificación y el algoritmo de resolución adecuados para un determinado problema?

No existe una respuesta “universal”, cada problema es diferente...

Actualmente, existe una amplia comunidad científica trabajando en problemas de razonamiento con restricciones, tanto para mejorar la eficiencia de los algoritmos de resolución (incluyendo las heurísticas), como para mejorar la codificación de los problemas.

Sin embargo, existen **técnicas generales** con las que podemos resolver razonablemente bien un gran número de CSP.

¿Qué veremos en esta práctica?

Codificar problemas usando un lenguaje de CP

- Los resolveremos usando CP solvers (y sus correspondientes heurísticas) como “caja negra” (*black box*).
- Una de las claves está en encontrar una **buena representación del problema** (variables, dominios) que nos permitan fácilmente expresar las restricciones.
- Además, una buena representación nos permitirá que el CP solver sea capaz de resolver el problema **más eficientemente**.

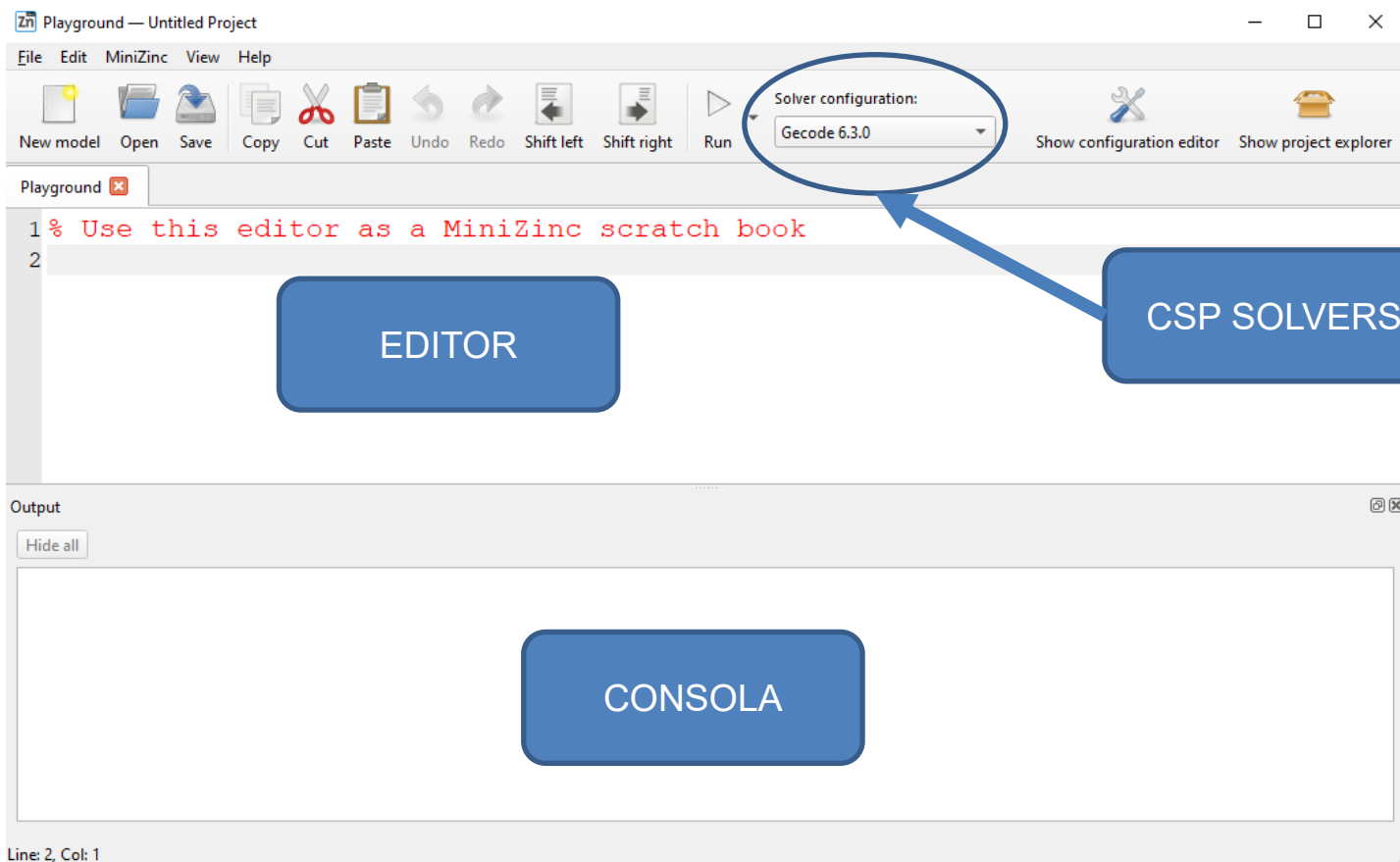
¿Qué **NO** veremos en esta práctica?

Algoritmos de resolución (búsqueda) y **heurísticas** como “caja blanca” (*white box*).

- En CSP/COP **complejos**, estos conocimientos ayudan a elegir la codificación y algoritmos de resolución más eficientes.

En esta práctica usaremos **MiniZinc**: <https://www.minizinc.org/>

MiniZinc es un **lenguaje de modelado de restricciones** gratuito y *open-source*. Su distribución es multiplataforma (Windows, Linux, macOS) e incluye un IDE con un editor de CSP y una batería de CSP *solvers* (ampliable) para resolverlos.



Materiales de apoyo:

Tutorial:

https://docs.minizinc.dev/en/stable/part_2_tutorial.html

User manual:

https://docs.minizinc.dev/en/stable/part_3_user_manual.html

Reference manual:

https://docs.minizinc.dev/en/stable/part_4_reference.html

Handbook: <https://docs.minizinc.dev/en/stable/>

(PDF: https://docs.minizinc.dev/_downloads/en/stable/pdf/)

MiniZinc es un **lenguaje de programación declarativa** (véase <https://docs.minizinc.dev/en/stable/spec.html>):

- Definimos variables, constantes y restricciones, pero **no definimos explícitamente la secuencia de pasos necesaria para encontrar la solución**. Dejamos que el *solver* satisfaga esas restricciones para encontrar la solución.
- En otras palabras, **nos centramos en codificar el resultado que buscamos y no los pasos para llegar ahí**.
 - El lenguaje describe el resultado deseado y no los cálculos directamente.

Ejemplo de programación imperativa (Java)

```
List<Customer> customers = new ArrayList<>();
customers.add(new Customer("John", "USA"));
customers.add(new Customer("Alice", "Canada"));
customers.add(new Customer("Bob", "USA"));
customers.add(new Customer("Eva", "USA"));

List<Customer> usaCustomers = new ArrayList<>();
for (Customer customer : customers) {
    if ("USA".equals(customer.getCountry())) {
        usaCustomers.add(customer);
    }
}
```

Ejemplo de programación declarativa (SQL)

```
SELECT * FROM Customers WHERE Country = 'USA';
```

<https://ondrej-kvasnovsky.medium.com/constraint-programming-ac8b4f49cdd2>

La distribución de MiniZinc se puede encontrar en:

<https://www.minizinc.org/downloads/>

Instalaremos la distribución apropiada a nuestro sistema operativo.

<https://docs.minizinc.dev/en/stable/installation.html>

Crearemos nuestro **primer problema CSP**:

% Definición de variables:

var int: x;

var int: y;

% Definición de restricciones:

constraint x > 0;

constraint y > 0;

constraint x > y;

% Indica que es un CSP (podría ser un COP)

solve satisfy;

El sistema va a intentar asignar valores a las variables, de modo que se cumplan las restricciones.

p.ej. solve minimize x + y;

Ejecutamos con el botón **"Run"** (empleando el CSP *solver* por defecto: Gecode 6.3.0), y vemos el resultado en la **Consola**.

Playground — Untitled Project*

File Edit MiniZinc View Help

New model Open Save Copy Cut Paste Undo Redo Shift left Shift right Run Solver configuration: Gecode 6.3.0 *

Hide configuration editor Show project explorer

```

1 % Definición de variables:
2 var int: x;
3 var int: y;
4 % Definición de restricciones:
5 constraint x > 0;
6 constraint y > 0;
7 constraint x > y;
8 % Indica que es un CSP (podría ser un COP)
9 solve satisfy;
10

```

Configuration

Configuration: Gecode 6.3.0 * Clone Set as default

Solver: Gecode 6.3.0 Reset to defaults

Options

☒ Maintain these options across solver configurations

Solving

☐ Time limit: 0,000s

☐ Default behavior ☒ User defined behavior

Optimization problems

☒ Print intermediate solutions

☒ Stop after this many optimal solutions (uncheck for all)

1

Satisfaction problems

☒ Stop after this many solutions (uncheck for all)

1

Output

☐ Verbose compilation ☐ Verbose Solving

☐ Output compilation statistics ☐ Output solving statistics

☐ Output timing information

Advanced options

Compiling

Optimization level: -O0 (no optimization)

Output

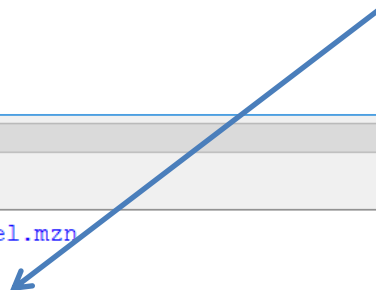
Running untitled_model.mzn 191msec

x = 2;
y = 1;

Finished in 191msec.

Line: 10, Col: 1

MiniZinc, por defecto,
devuelve la primera
solución que encuentra.



```
solve satisfy;  
solve maximize <arithmetic expression>;  
solve minimize <arithmetic expression>;
```

Crearemos nuestro **primer problema COP**:

% Definición de variables:

var 2..10: x;



Dominio restringido

var 2..10: y;

% Definición de restricciones:

constraint x mod 2 == 0;



Valores múltiplos de 2 y 3 respectivamente

constraint y mod 3 == 0;

constraint (x*y) mod 5 == 0;



Cuyo producto es múltiplo de 5

% Indica COP

solve maximize x + y;



¿Cuáles son los mayores valores que satisfacen estas restricciones?

Ejecutamos con el botón **"Run"** y vemos el resultado en la **Consola**.

% ESTRUCTURA GENERAL DE UN CSP EN MINIZINC

% 1. Definición de variables y constantes:

```
var int : x;
```

Enteros en [-1, 1]

```
var -1..1 : y;
```

```
int : z = 6;
```

Constante z con valor 6. Las constantes en MiniZinc se llaman *parameters* (vs *decision variables*):

<https://docs.minizinc.dev/en/stable/spec.html>

% 2. Definición de restricciones:

```
constraint x > 0 /\ y > 0;
```

```
constraint x > y;
```

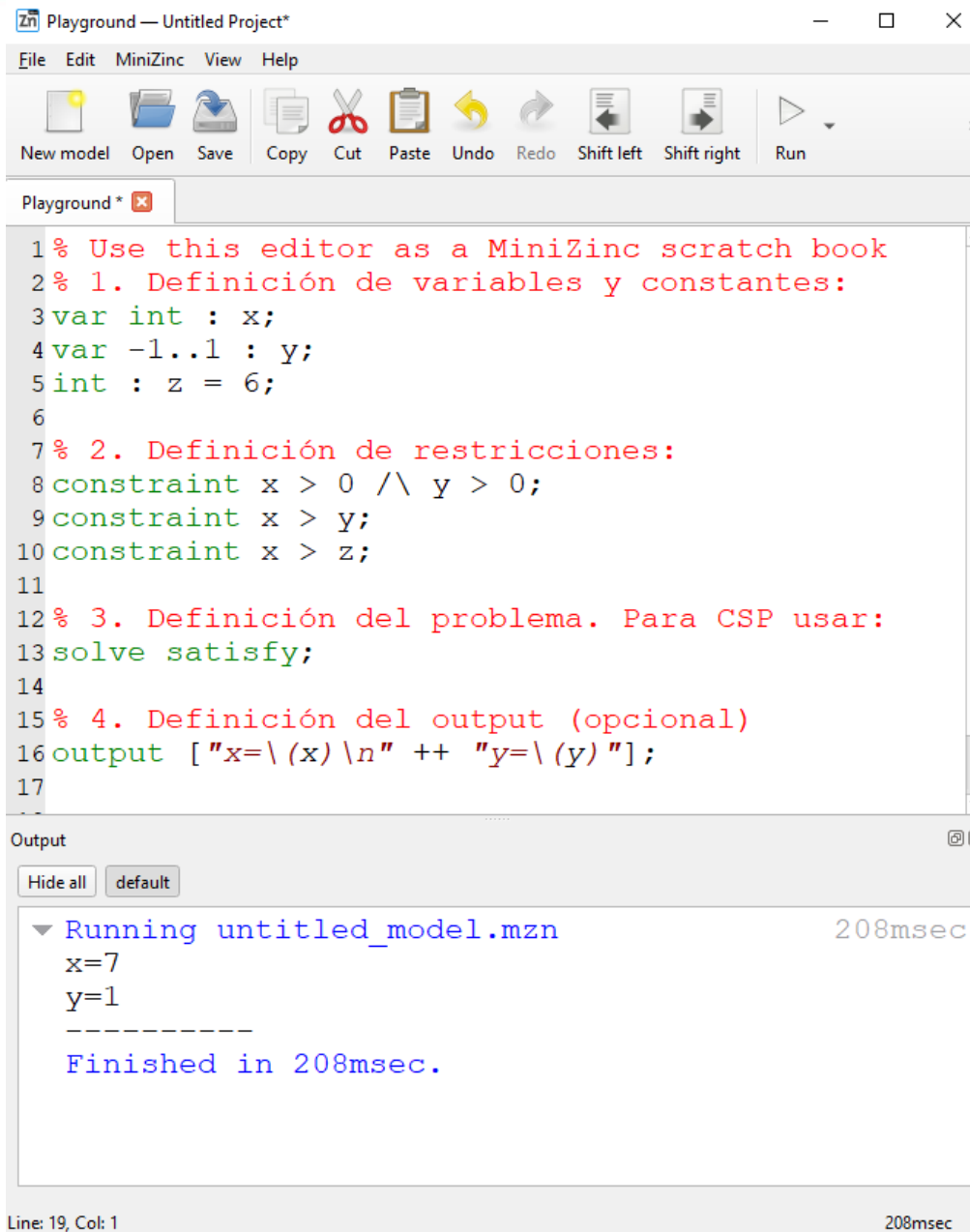
```
constraint x > z;
```

% 3. Definición del problema. Para CSP usar:

```
solve satisfy;
```

% 4. Definición del output (opcional)

```
output ["x=\(x)\n" ++ "y=\(y)"];
```



The screenshot shows the MiniZinc Playground interface. The main editor contains a MiniZinc model with the following code:

```

1 % Use this editor as a MiniZinc scratch book
2 % 1. Definición de variables y constantes:
3 var int : x;
4 var -1..1 : y;
5 int : z = 6;
6
7 % 2. Definición de restricciones:
8 constraint x > 0 /\ y > 0;
9 constraint x > y;
10 constraint x > z;
11
12 % 3. Definición del problema. Para CSP usar:
13 solve satisfy;
14
15 % 4. Definición del output (opcional)
16 output [ "x=\ (x) \n" ++ "y=\ (y) " ];
17

```

The output window shows the result of running the model:

```

Running untitled_model.mzn 208msec
x=7
y=1
-----
Finished in 208msec.

```

The status bar at the bottom indicates "Line: 19, Col: 1" and "208msec".

% Las variables son aquellas para las que el CSP *solver* va a buscar valores dentro de sus dominios y respetando las restricciones

% La definición de toda variable comienza por "var"

% seguida por su dominio y su nombre o identificador:

```
var int : x;
```

```
var -1..1 : y;
```

% Un dominio puede ser:

% * Un tipo primitivo (int, float o bool)

% * Un intervalo (para int: "a .. b"; para float: "a.x .. b.y", p.ej. 0.0 .. 10.0)

% * Un conjunto previamente definido ("set of"):

```
set of int: values1 = -1..1;
```

```
var values1: y2;
```

```
set of int: values2 = {-1,0,1};
```

```
var values2: y3;
```

% Las constantes son datos del problema que participan en las restricciones del problema, pero cuyos valores son fijos

% Se definen igual que las variables, pero sin la palabra reservada "var". Se puede tanto declarar su valor como no declararlo:

```
int : z1;
```

```
int : z2 = 6;
```

% Si no se define su valor, éste será preguntado antes de comenzar la ejecución.

Ejemplo:

```
var int : x;
```

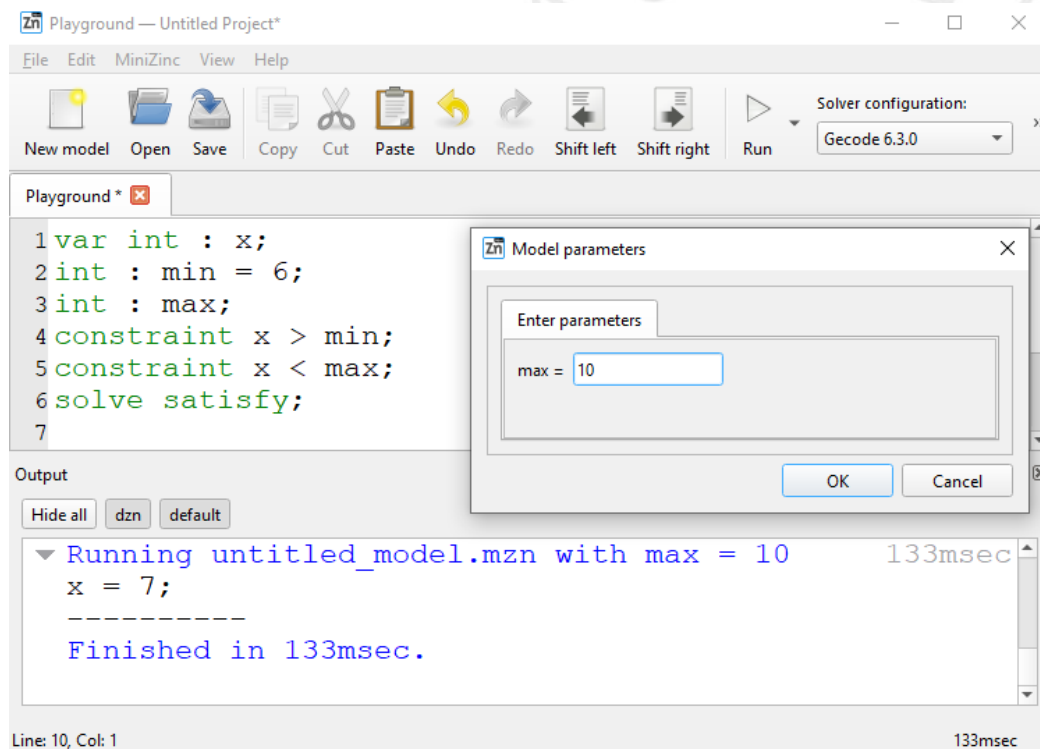
```
int : min = 6;
```

```
int : max;
```

```
constraint x > min;
```

```
constraint x < max;
```

```
solve satisfy;
```



% La definición de una restricción comienza por "constraint"

% Se pueden usar los operadores lógicos/aritméticos habituales:

% $\leq$, $<$, $=$, $\neq$, $>$, $\geq$

% $+$, $-$, $*$. División entera "a div b". Resto "a mod b"

% AND: " $\wedge$ " OR: " $\vee$ " IF: " $\rightarrow$ " IFF: " $\leftrightarrow$ "

% Ejemplo:

var 1..3: WA; var 1..3: NT; var 1..3: SA; var 1..3: Q;

var 1..3: NSW; var 1..3: V; var 1..3: T;

constraint WA $\neq$ NT $\wedge$ WA $\neq$ SA ;

constraint NT $\neq$ SA $\wedge$ NT $\neq$ Q ;

constraint Q $\neq$ NSW ;

constraint NSW $\neq$ V ;

constraint SA $\neq$ Q ;

constraint SA $\neq$ NSW $\wedge$ SA $\neq$ V ;

solve satisfy;



Playground — Untitled Project*

File Edit MiniZinc View Help

New model Open Save Copy Cut Paste Undo Redo Shift left Shift right Run Solver configuration: Gecode 6.3.0

```

1 var 1..3: WA; var 1..3: NT; var 1..3: SA; var 1..3: Q;
2 var 1..3: NSW; var 1..3: V; var 1..3: T;
3
4 constraint WA != NT /\ WA != SA ;
5 constraint NT != SA /\ NT != Q ;
6 constraint Q != NSW ;
7 constraint NSW != V ;
8 constraint SA != Q ;
9 constraint SA != NSW /\ SA != V ;
10
11 solve satisfy;
12

```

Output

Hide all dzn default

Running untitled_model.mzn

```

WA = 3;
NT = 2;
SA = 1;
Q = 3;
NSW = 2;
V = 3;
T = 1;
-----
Finished in 130msec.

```

Line: 14, Col: 1 130msec



% Una forma muy útil de definir variables es mediante **arrays**, los cuales son a menudo el input de algunas restricciones globales.

% Un vector (unidimensional) se define como:

```
array [<set-of-int>] of var <type>: <identifier>;
```

% donde **<set-of-int>** **identifica las posiciones del array**, y puede ser cualquier conjunto de enteros (también negativos), y **<type>** **determina el tipo de cada posición en el array**. Ejemplo:

```
array [1..7] of var 1..3: regiones;
```

```
constraint regiones[1] != regiones[2] /\ regiones[1] != regiones[3];
```

```
constraint regiones[2] != regiones[3] /\ regiones[2] != regiones[4];
```

```
constraint regiones[4] != regiones[5];
```

```
constraint regiones[5] != regiones[6] ;
```

```
constraint regiones[3] != regiones[4] /\ regiones[3] != regiones[5] /\  
regiones[3] != regiones[6] ;
```

```
solve satisfy;
```

```
output ["region \ (i) = \ (regiones[i]) \ n" | i in 1..7];
```

% El conjunto de posiciones puede ser cualquiera (ipero cuidado cuando se accede a él!):

```
array [1..7] of var 1..3: regiones;
    output ["\"(regiones[i]) \" | i in 1..7];
array [10..16] of var 1..3: regiones;
    output ["\"(regiones[i]) \" | i in 10..16];
array [-10..-4] of var 1..3: regiones;
    output ["\"(regiones[i]) \" | i in -10..-4];
```

% Para evitarlo se pueden definir "sets"

```
set of int: POS = 1..7;
set of int: VALUES = 1..3;
array [POS] of var VALUES: regiones;
output ["\"(regiones[i]) \" | i in POS];
```

% También se pueden definir *arrays* de constantes

% (sin la palabra "var"):

```
set of int: POS = 1..7;
```

```
set of int: VALUES = 1..3;
```

```
array [POS] of var VALUES: regiones;
```

```
array[POS] of string: strRegiones =
```

```
["WA","NT", "SA", "Q", "NSW", "V", "T"];
```

```
constraint regiones[1] != regiones[2] /\ regiones[1] != regiones[3] ;
```

```
constraint regiones[2] != regiones[3] /\ regiones[2] != regiones[4] ;
```

```
constraint regiones[4] != regiones[5] ;
```

```
constraint regiones[5] != regiones[6] ;
```

```
constraint regiones[3] != regiones[4] ;
```

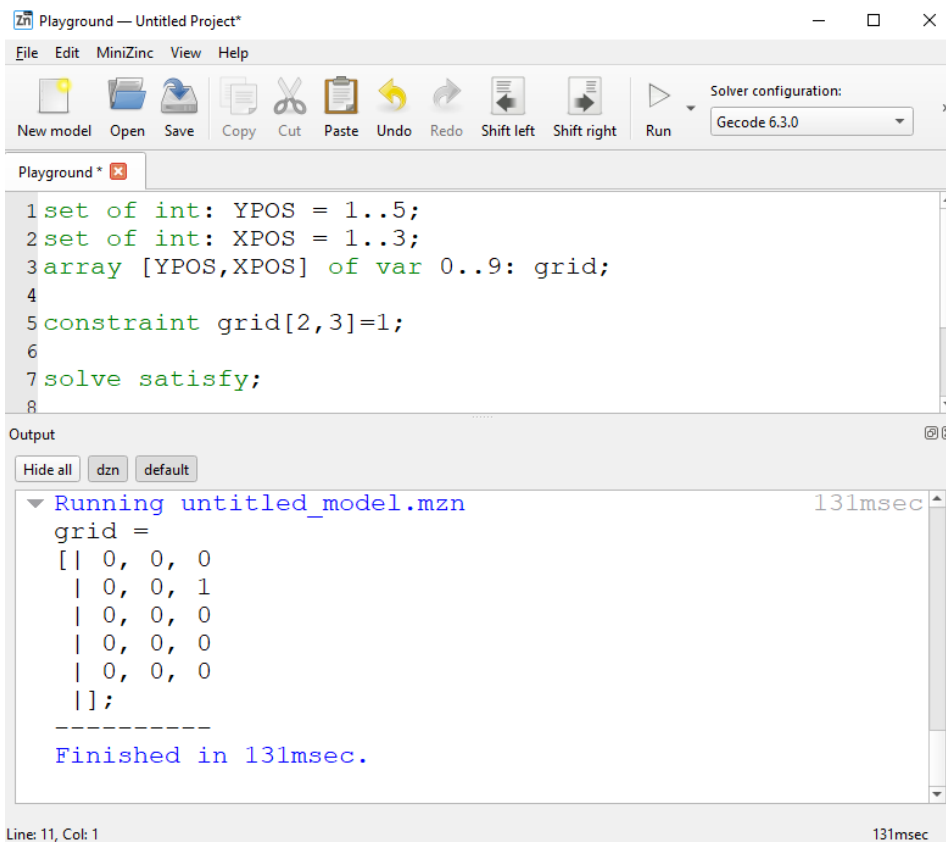
```
constraint regiones[3] != regiones[5] /\ regiones[3] != regiones[6] ;
```

```
output ["\\(strRegiones[i]) = \\\(regiones[i])\\n" | i in POS];
```

% Para definir un *array* multidimensional únicamente tenemos que
% definir los tamaños de cada dimensión:

```
set of int: YPOS = 1..5;
set of int: XPOS = 1..3;
array [YPOS,XPOS] of var 0..9: grid;
% Grid de 5 filas, 3 columnas,
con valores del 0 al 9:
```

grid	x=1	x=2	x=3
y=1	var 0..9	var 0..9	var 0..9
y=2	var 0..9	var 0..9	var 0..9
y=3	var 0..9	var 0..9	var 0..9
y=4	var 0..9	var 0..9	var 0..9
y=5	var 0..9	var 0..9	var 0..9



The screenshot shows the MiniZinc Playground interface. The code editor contains the following code:

```
1 set of int: YPOS = 1..5;
2 set of int: XPOS = 1..3;
3 array [YPOS,XPOS] of var 0..9: grid;
4
5 constraint grid[2,3]=1;
6
7 solve satisfy;
```

The output window shows the result of solving the model:

```
Running untitled_model.mzn 131msec
grid =
[| 0, 0, 0
 | 0, 0, 1
 | 0, 0, 0
 | 0, 0, 0
 | 0, 0, 0
 |];
-----
Finished in 131msec.
```

% Otra estructura de datos útil son los **"enum"** que, como en otros lenguajes, son conjuntos ordenados con valores únicos y accesibles por su nombre:

```
enum Tarea = {A,B,C};
enum Trabajadores = {T1,T2,T3};
array [Tarea] of set of Tarea:
    Predecesoras = [{},{A},{A,B}];
```

Permiten definir
tamaños como un set

```
array [Tarea] of var Trabajadores:
    asignacion;
constraint asignacion[A] == T1;
constraint asignacion[B] == T3 ∨ asignacion[B] == T2 ;
```

Pueden referirse por su
nombre

Pueden utilizarse como
índice o clave

```
output["\\(t):\\(asignacion[t]) \\n" | t in Tarea];
```

A:T1
B:T2
C:T1

<https://docs.minizinc.dev/en/stable/modelling2.html> (sección *Enumerated Types*)

% ¿Se puede declarar un array en MiniZinc con **dominio diferente en cada casilla**? Sí, indirectamente. En cualquier caso, en MiniZinc **todos los elementos de un array deben tener el mismo tipo base**.

```
set of int: I = 1..4;
```

```
array[I] of int: lb = [1,4,2,7];
```

```
array[I] of int: ub = [3,8,5,9];
```

```
array[I] of var 1..9: x;
```

```
constraint forall(i in I)(
  x[i] >= lb[i] /\ x[i] <= ub[i]
);
```

```
solve satisfy;
```

```
output [ "x = " ++ show(x) ];
```

x = [1, 4, 2, 7]

```
set of int: I = 1..4;
```

```
array[I] of set of int: dom = [
  1..3,
  4..8,
  2..5,
  7..9
];
```

```
array[I] of var 1..9: x;
```

```
constraint forall(i in I)(
  x[i] in dom[i]
);
```

```
solve satisfy;
```

```
output [ "x = " ++ show(x) ];
```

- El formateo de salida (**output**) debe ser una **lista de strings**.
- Algunas funciones útiles para mostrar el valor de las variables: **show**, **show_int**, **show_float**.

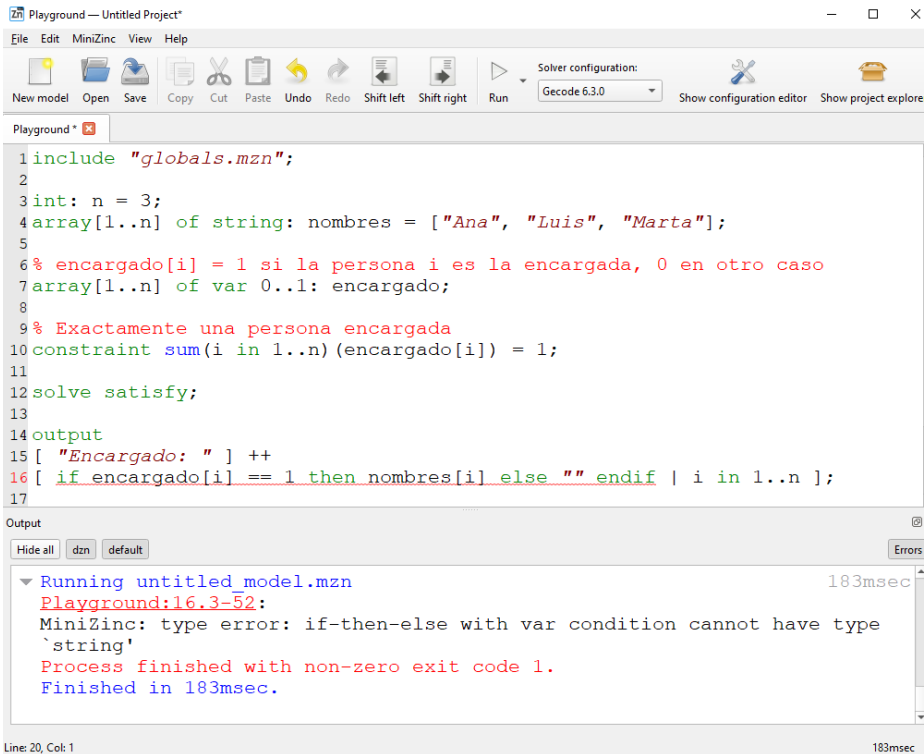
output["\n(S)\n"]
output[show(S)]

Expresiones equivalentes

output["t=\n(t)\n"]
output["t=" ++ show(t) ++ "\n"]

Expresiones equivalentes

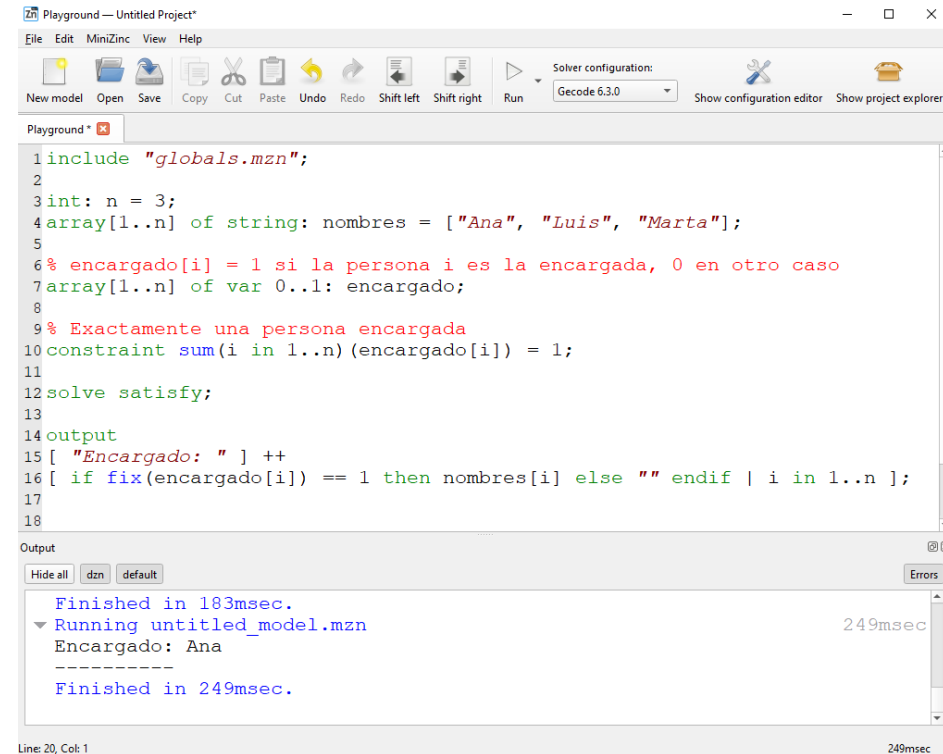
- fix** se utiliza para recuperar el valor asignado a una variable y usarlo en expresiones dentro del *output*:

```
1 include "globals.mzn";
2
3 int: n = 3;
4 array[1..n] of string: nombres = ["Ana", "Luis", "Marta"];
5
6 % encargado[i] = 1 si la persona i es la encargada, 0 en otro caso
7 array[1..n] of var 0..1: encargado;
8
9 % Exactamente una persona encargada
10 constraint sum(i in 1..n) (encargado[i]) = 1;
11
12 solve satisfy;
13
14 output
15 [ "Encargado: " ] ++
16 [ if encargado[i] == 1 then nombres[i] else "" endif | i in 1..n ];
17
```

Output

Running untitled_model.mzn
Playground:16.3-52:
MiniZinc: type error: if-then-else with var condition cannot have type `string`
Process finished with non-zero exit code 1.
Finished in 183msec.



```
1 include "globals.mzn";
2
3 int: n = 3;
4 array[1..n] of string: nombres = ["Ana", "Luis", "Marta"];
5
6 % encargado[i] = 1 si la persona i es la encargada, 0 en otro caso
7 array[1..n] of var 0..1: encargado;
8
9 % Exactamente una persona encargada
10 constraint sum(i in 1..n) (encargado[i]) = 1;
11
12 solve satisfy;
13
14 output
15 [ "Encargado: " ] ++
16 [ if fix(encargado[i]) == 1 then nombres[i] else "" endif | i in 1..n ];
17
18
```

Output

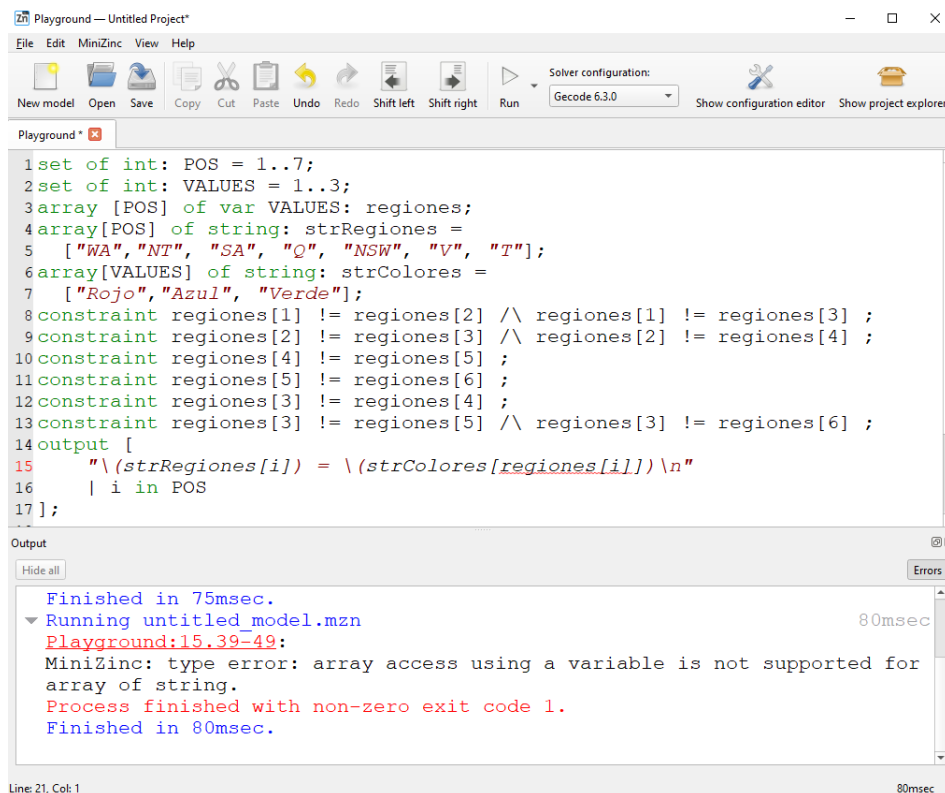
Finished in 183msec.
Running untitled_model.mzn
Encargado: Ana

Finished in 249msec.

En el ejemplo que se muestra en esta diapositiva el problema es que:
encargado[i] == 1
 no es un **bool**, sino un **var bool**, y necesitamos instanciarlo para usarlo en el **if**.

<https://docs.minizinc.dev/en/stable/modelling.html> (sección *Output and Strings*)

- fix** se utiliza para recuperar el valor asignado a una variable y usarlo en expresiones dentro del *output*:

```

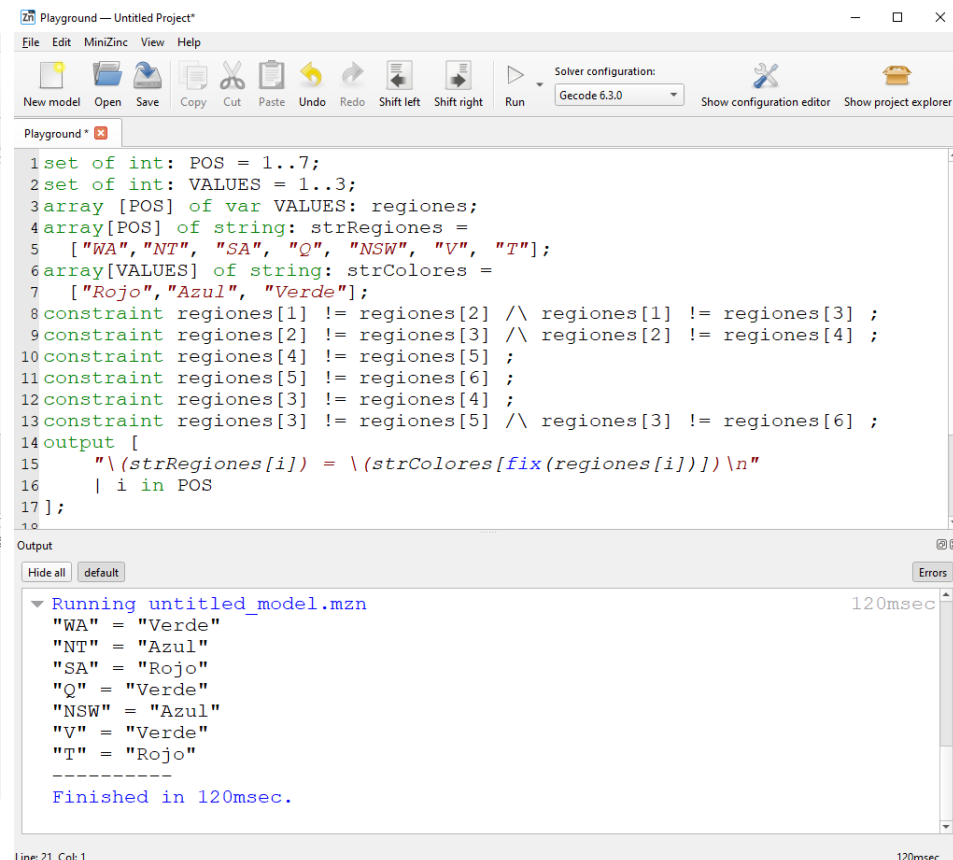
1 set of int: POS = 1..7;
2 set of int: VALUES = 1..3;
3 array [POS] of var VALUES: regiones;
4 array [POS] of string: strRegiones =
5   ["WA", "NT", "SA", "Q", "NSW", "V", "T"];
6 array [VALUES] of string: strColores =
7   ["Rojo", "Azul", "Verde"];
8 constraint regiones[1] != regiones[2] /\ regiones[1] != regiones[3] ;
9 constraint regiones[2] != regiones[3] /\ regiones[2] != regiones[4] ;
10 constraint regiones[4] != regiones[5] ;
11 constraint regiones[5] != regiones[6] ;
12 constraint regiones[3] != regiones[4] ;
13 constraint regiones[3] != regiones[5] /\ regiones[3] != regiones[6] ;
14 output [
15   "\ (strRegiones[i]) = \ (strColores[regiones[i]]) \n"
16   | i in POS
17 ];
  
```

Output

```

Finished in 75msec.
Running untitled_model.mzn 80msec
Playground:15.39-49:
MiniZinc: type error: array access using a variable is not supported for
array of string.
Process finished with non-zero exit code 1.
Finished in 80msec.
  
```

Line: 21, Col: 1



```

1 set of int: POS = 1..7;
2 set of int: VALUES = 1..3;
3 array [POS] of var VALUES: regiones;
4 array [POS] of string: strRegiones =
5   ["WA", "NT", "SA", "Q", "NSW", "V", "T"];
6 array [VALUES] of string: strColores =
7   ["Rojo", "Azul", "Verde"];
8 constraint regiones[1] != regiones[2] /\ regiones[1] != regiones[3] ;
9 constraint regiones[2] != regiones[3] /\ regiones[2] != regiones[4] ;
10 constraint regiones[4] != regiones[5] ;
11 constraint regiones[5] != regiones[6] ;
12 constraint regiones[3] != regiones[4] ;
13 constraint regiones[3] != regiones[5] /\ regiones[3] != regiones[6] ;
14 output [
15   "\ (strRegiones[i]) = \ (strColores[fix(regiones[i])]) \n"
16   | i in POS
17 ];
  
```

Output

```

Running untitled_model.mzn 120msec
"WA" = "Verde"
"NT" = "Azul"
"SA" = "Rojo"
"Q" = "Verde"
"NSW" = "Azul"
"V" = "Verde"
"T" = "Rojo"
-----
Finished in 120msec.
  
```

Line: 21, Col: 1

% Es común usar **restricciones predefinidas** en librerías que representan abstracciones de modelado de alto nivel. **Muchos solvers implementan algoritmos de inferencia específicos y eficientes para estas restricciones** (<https://docs.minizinc.dev/en/stable/lib-globals.html>). A continuación, se incluyen algunas de las más comunes:

```
include "globals.mzn";
constraint all_different(<array>);
constraint forall(<set>)(<restricción>);
```

% Por ejemplo:

```
array[0..9] of var 0..9: digitos;
constraint all_different(digitos);
constraint forall(i,j in 0..9)(i!=j -> digitos[i] != digitos[j]);
constraint forall(i,j in 0..9)(i==j ∨ digitos[i] != digitos[j]);
constraint forall(i in 0..9)(forall(j in 0..9)(i==j ∨ digitos[i] != digitos[j]));
```

$$a \rightarrow b \equiv \neg a \vee b$$

$$i \neq j \rightarrow \text{digitos}[i] \neq \text{digitos}[j]$$

% all_different se puede aplicar a array multidimensionales:

```
array[1..3,1..3] of var 1..9: grid;
constraint all_different( [ grid[i,j] | i,j in 1..3 ] );
constraint all_different( grid );
```

iAmbos all_different producen el mismo resultado!

% **También se pueden usar predicados**

definidos por el usuario (["a predicate is just a function with output type var bool"](#))

```
include "globals.mzn";
```

```
int: N = 10;
```

```
int: maxValue = 100;
```

```
array[1..N] of var 0..maxValue: vector;
```

% Todos los valores del vector deben ser diferentes excepto el 0

```
constraint all_different_except_0(vector);
```

```
constraint sum(i in 1..N)(vector[i]) < 10 /\ sum(i in 1..N)(vector[i]) > 5;
```

```
solve satisfy;
```

```
vector = [6, 0, 0, 0, 0, 0, 0, 0, 0, 0];  
-----
```

iSi usamos `all_different`
el resultado sería
insatisfactible!

```
include "globals.mzn";
```

```
int: N = 5;
```

```
int: maxValue = 100;
```

```
set of int: exception = {2};
```

```
array[1..N] of var 1..maxValue: vector;
```

% Todos los valores del vector deben ser diferentes excepto los que proporcionamos como excepción

```
constraint all_different_except(vector, exception);
```

```
constraint sum(i in 1..N)(vector[i]) < 11 /\ sum(i in 1..N)(vector[i]) > 5;
```

```
solve satisfy;
```

```
vector = [2, 2, 2, 2, 1];
```

```
-----
```

% Debemos habituarnos a pensar de modo declarativo. Por ejemplo, **¿de qué manera podríamos hacer un “find” en MiniZinc?** Es decir, queremos encontrar la posición de un valor dado en un vector determinado.

```
var int: pos; % posición en que se encuentra 'target_value' en 'vector'
```

```
int: n = 10; % tamaño del vector
```

```
int: target_value; % valor cuya posición se desea conocer.
```

```
array[1..n] of int: vector = [100,20,30,4,50,6,70,8,90,10];
```

```
constraint vector[pos]=target_value;
```

```
solve satisfy;
```

% Todas estas restricciones se pueden combinar, como es lógico, con otras funciones típicas (suma, contar, etc.):

sum(<POS>) (<array>)

count(<array>, <value>)

...

% Ejemplo:

% Array con 10 posiciones con números del 0 al 20

array[0..9] of var 0..20: myarray;

% All different:

constraint forall(i in 0..20)(count(myarray,i)<=1);

% La suma de la primera mitad igual a la suma de la segunda mitad

constraint sum(i in 0..(length(myarray) div 2)-1)(myarray[i]) ==
sum(i in (length(myarray) div 2)..9)(myarray[i]);

solve satisfy;

% **Acordaos de revisar la documentación con calma.** Por ejemplo, en <https://docs.minizinc.dev/en/stable/lib-globals-counting.html> podéis ver que hay distintas variantes de **count**(<array>,<value>)

% Ejemplo:

```
include "globals.mzn";
```

```
int: N = 5;
```

```
int: maxValue = 100;
```

```
array[1..N] of var 1..maxValue: vector;
```

% Debemos encontrar un vector de tamaño 5, de valores entre 1 y 100, cuya suma sea >5 y <10, y en donde los dígitos 1 y 2 tienen que aparecer 2 veces cada uno.

```
constraint count(vector, 1, 2);
```

```
constraint count(vector, 2, 2);
```

```
constraint sum(i in 1..N)(vector[i]) < 10 /\ sum(i in 1..N)(vector[i]) > 5;
```

```
solve satisfy;
```

```
predicate count(array [int] of var int: x, var int: y, var int: c)
```

Constrains **c** to be the number of occurrences of **y** in **x**.

```
vector = [3, 2, 1, 1, 2];
```

```
include "globals.mzn";
```

```
int: N = 5;
```

```
int: maxValue = 100;
```

```
array[1..N] of var 1..maxValue: vector;
```

```
% Los valores del vector son todos diferentes
```

```
constraint all_different(vector);
```

```
% Son todos pares
```

```
constraint forall(i in 1..N)((vector[i] mod 2) == 0);
```

```
% Existen dos valores cuya suma da el valor máximo
```

```
constraint exists(i,j in 1..N)(j != i /\ (vector[i]+vector[j] == maxValue));
```

```
solve satisfy;
```

```
output [ "\ (vector[i]) " | i in 1..N ] ++ ["\n"];
```

92	8	6	4	2

% Queremos un vector de 5 elementos, en donde todos sean diferentes,
y en el que la suma de las posiciones pares sea igual al de las impares
include "globals.mzn";

```
int: n = 5;
```

```
array[1..n] of var 1..6: vector;
```

```
constraint alldifferent(vector);
```

```
set of int: pares = {i | i in 1..n where (i mod 2 = 0)};
```

```
set of int: impares = {i | i in 1..n where (i mod 2 != 0)};
```

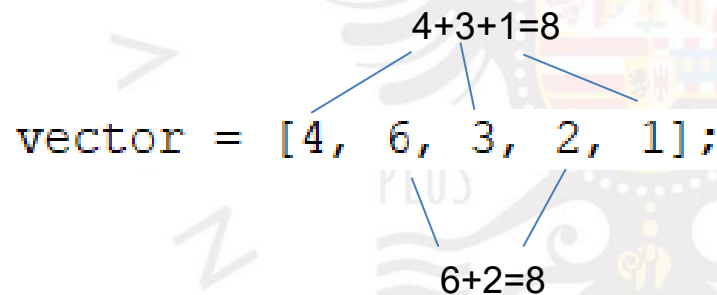
```
constraint sum(i in pares)(vector[i]) == sum(i in impares)(vector[i]);
```

```
solve satisfy;
```

vector = [4, 6, 3, 2, 1];

4+3+1=8

6+2=8



% Imaginemos que tenemos una matriz y nos queremos quedar con el máximo de entre todas las sumas de las columnas de dicha matriz. Es decir, **queremos la suma máxima por columna y la posición/columna en que se encuentra dicha suma máxima.**

```
int: nFilas = 3;
int: nColumnas = 7;
var 1..nColumnas : pos;
```

1	2	3	4	5	6	7
7	5	12	5	3	12	12
14	13	12	11	10	9	8

Suma por columna → 22 20 27 20 18 27 27

```
array [1..nFilas, 1..nColumnas] of int: valores = [| 1,2,3,4,5,6,7 |
7,5,12,5,3,12,12 | 14,13,12,11,10,9,8 |];
```

```
int: maximo = max(i in 1..nColumnas)(sum(j in 1..nFilas)(valores[j,i]));
```

```
constraint (sum(j in 1..nFilas)(valores[j,pos]))=maximo;
```

```
output["\\(maximo) en columna \\(pos)"]
```

27 en columna 3

COMANDO	PROPÓSITO	DOCUMENTACIÓN
<code>all_different</code>	Forzar a que todos los valores asignados a todas las variables en un array sean diferentes.	https://docs.minizinc.dev/en/stable/lib-globals-alldifferent.html#mzn-globals-alldifferent-all-different
<code>all_different_except_0</code>	Igual que <code>all_different</code> (salvo el 0, que puede aparecer repetido).	https://docs.minizinc.dev/en/stable/lib-globals-alldifferent.html#mzn-globals-alldifferent-all-different-except-0
<code>all_different_except</code>	Igual que <code>all_different</code> (salvo para el valor que se especifique, que puede aparecer repetido).	https://docs.minizinc.dev/en/stable/lib-globals-alldifferent.html#mzn-globals-alldifferent-all-different-except
<code>forall</code>	Forzar que se cumpla una restricción para todos los elementos de un conjunto. Conjunción lógica de las restricciones.	https://docs.minizinc.dev/en/stable/lib-stdlib-builtins.html#mzn-stdlib-builtins-logic-forall y https://docs.minizinc.dev/en/stable/modelling2.html#index-17
<code>exists</code>	Forzar que se cumpla una restricción para alguno de los elementos de un conjunto.	https://docs.minizinc.dev/en/stable/lib-stdlib-builtins.html#exists y https://docs.minizinc.dev/en/stable/modelling2.html#index-17
<code>sum</code>	Calcular la suma de los elementos de un array.	https://docs.minizinc.dev/en/stable/lib-stdlib-builtins.html#index-26
<code>count</code>	Devolver el número de elementos ciertos de un array.	https://docs.minizinc.dev/en/stable/lib-stdlib-builtins.html#count
<code>max</code>	Devolver el máximo de los elementos de un array.	https://docs.minizinc.dev/en/stable/lib-stdlib-builtins.html#index-19
<code>where</code>	Construir una lista en donde sus elementos cumplen una determinada condición booleana.	https://docs.minizinc.dev/en/stable/modelling2.html#defblock-3

Puzzle cripto-aritmético

Problema: cada letra codifica un único dígito (es decir, un número entero en $[0,9]$) y cada dígito está asignado a una única letra de forma que se satisfaga la operación aritmética (en este caso, la suma).

$$\text{SEND} + \text{MORE} = \text{MONEY}$$

$$(9567 + 1085 = 10652)$$

% Ejemplo: SEND-MORE-MONEY

include "alldifferent.mzn";

var 1..9: S; var 1..9: M;

var 0..9: E; var 0..9: O;

var 0..9: N; var 0..9: R;

var 0..9: D; var 0..9: Y;

Se asume que ningún número puede empezar por 0, porque entonces dejaría de ser un número de 4 o 5 cifras y pasaría a ser considerado un número de menos cifras.

constraint 1000 * S + 100 * E + 10 * N + D
+ 1000 * M + 100 * O + 10 * R + E =
10000 * M + 1000 * O + 100 * N + 10 * E + Y;

constraint alldifferent([S,E,N,D,M,O,R,Y]);

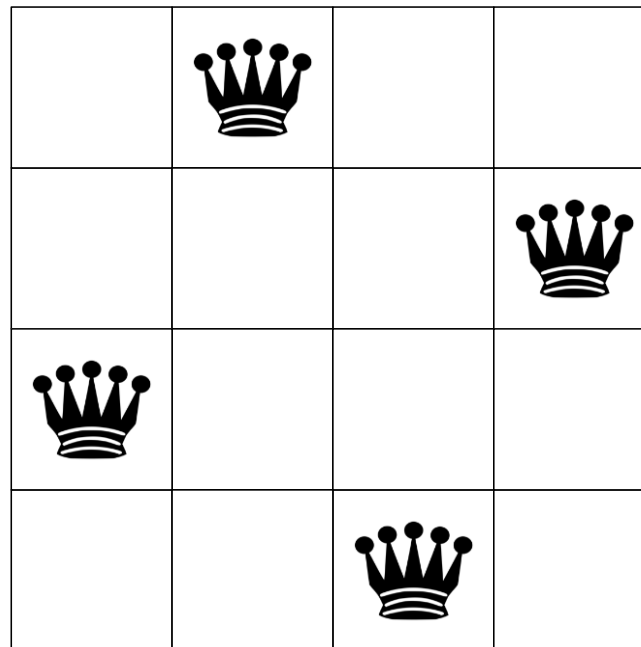
solve satisfy;

output ["\\(S)\\(E)\\(N)\\(D) + \\(M)\\(O)\\(R)\\(E) = \\(M)\\(O)\\(N)\\(E)\\(Y)\\n"];

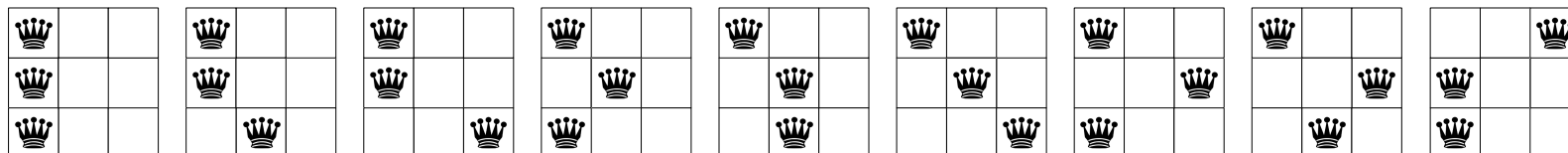
Problema de las N-Reinas

Problema: Posicionar N reinas en un tablero NxN de forma que ningún par de reinas se ataque mutuamente.

Ejemplo: 4-reinas



Problema de las N-Reinas

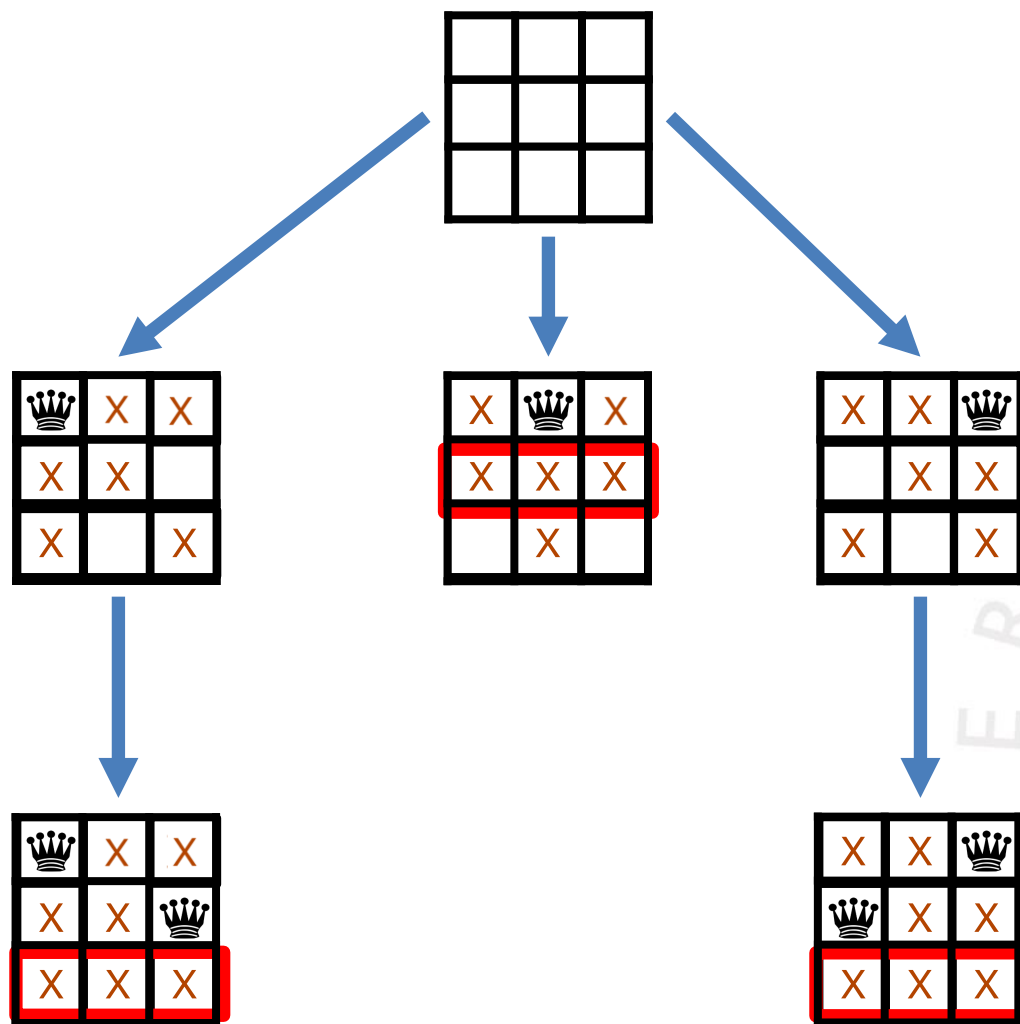


...

N	#configuraciones (N^N)
3	27
4	256
8	16.777.216
16	18.446.744.073.709.551.616
...	...

No se pueden explorar todas ellas por fuerza bruta.
¡Hace falta **búsqueda**!

Problema de las 3-Reinas



⇒ **UNSAT**

La búsqueda es mucho más eficiente que la fuerza bruta.

En lugar de explorar las 27 configuraciones existentes, nos basta con explorar 5 de ellas.

% Ejemplo: N-Reinas:

include "globals.mzn";

int: n;

array [1..n] of var 1..n: reinas;

	X=1	X=2	X=3	X=4
Y=1				
Y=2				
Y=3				
Y=4				

Fila1 Fila2 Fila3 Fila4

reinas = [2,4,1,3]

Esta codificación, implícitamente, ya impide la existencia de ataques en una misma fila. ¡Cada reina ya está en filas distintas gracias al modelado del problema!

```
% Ataques en misma fila: no hay (cada reina ya está en filas
% distintas gracias al modelado del problema)
constraint alldifferent(reinas); % Ataques en columnas

constraint alldifferent([reinas[i] - i | i in 1..n]); % Ataques en diag. princ.

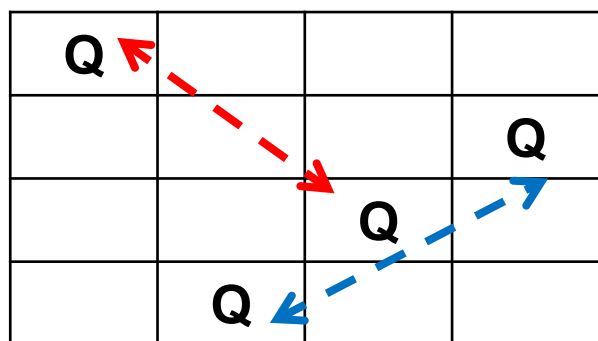
constraint alldifferent([reinas[i] + i | i in 1..n]); % Ataques en diag. sec.

solve satisfy;

output [ if fix(reinas[j]) == i then "Q" else "." endif ++
        if j == n then "\n" else "" endif | i,j in 1..n];
```

`constraint alldifferent([reinas[i] - i | i in 1..n]);` % Ataques en diag. princ.

`constraint alldifferent([reinas[i] + i | i in 1..n]);` % Ataques en diag. sec.



Fila2: Col4

Fila3: Col3

reinas = [1,4,3,2]

Fila1: Col1

Fila4: Col2

$$[1-1, 4-2, 3-3, 2-4] = [0, 2, 0, -2]$$

$$[1+1, 4+2, 3+3, 2+4] = [2, 6, 6, 6]$$

Problema del cuadrado mágico

Problema: Dado un tablero $N \times N$, posicionar los números entre 1 y N^2 de forma que todos ellos sean distintos y que todas las sumas por filas, columnas y diagonales principales sean iguales (esta suma se conoce como “suma mágica”).

Ejemplo: Cuadrado mágico de tamaño 4
(suma mágica = 34)

9	6	11	8
5	10	7	12
4	3	14	13
16	15	2	1

```
include "globals.mzn";
int: n;
set of int: S = 1..n; % size
set of int : N = 1 .. n*n; % numbers
array[S, S] of var N: magic;
var int: ms; % magic sum
```

También valdría
alldifferent(magic)

% Todos los números distintos

```
constraint alldifferent( [ magic[fila,col] | fila,col in S ] );
```

% Suma mágica en filas y columnas

```
constraint forall(fila in S)(sum(col in S) (magic[fila,col]) == ms);
```

```
constraint forall(col in S) (sum(fila in S) (magic[fila,col]) == ms);
```

% Suma mágica en diagonales

```
constraint sum(fila in S)(magic[fila,fila]) == ms;
```

```
constraint sum(fila in S) (magic[fila, n-fila+1]) == ms;
```

```
solve satisfy;
```

Problema del Sudoku

Problema: Rellenar una cuadrícula 9x9 (dividida en 9 subcuadrículas 3x3), parcialmente completada, con dígitos del 1 al 9 de forma que: (i) no se repita ningún dígito por filas, (ii) no se repita ningún dígito por columnas, y (iii) no se repita ningún dígito por subcuadrícula.

5	3			7				
6			1	9	5			
	9	8					6	
8				6				3
4			8		3			1
7				2				6
	6					2	8	
			4	1	9			5
				8			7	9

% Ejemplo: Sudoku

```
include "globals.mzn";
```

```
int: S = 3; int: N = S * S;
```

```
set of int: RangoPuzzle = 1..N; set of int: RangoCuadrado = 1..S;
```

```
array[RangoPuzzle, RangoPuzzle] of var RangoPuzzle: puzzle;
```

% Cada fila tiene todos los elementos diferentes

```
constraint forall (i in RangoPuzzle)
```

```
    ( alldifferent( [ puzzle[i,j] | j in RangoPuzzle ] ) );
```

% Cada columna tiene todos los elementos diferentes

```
constraint forall (j in RangoPuzzle)
```

```
    ( alldifferent( [ puzzle[i,j] | i in RangoPuzzle ] ) );
```

% (continua en la siguiente diapositiva...)

```
% int: S = 3; int: N = S * S;
```

```
% set of int: RangoPuzzle = 1..N; set of int: RangoCuadrado = 1..S;
```

```
%Todos diferentes en cada cuadrado
```

```
constraint forall (a, o in RangoCuadrado)
```

```
( alldifferent( [ puzzle[S*(a-1)+x , S*(o-1)+y]  
  | x, y in RangoCuadrado ] ) );
```

Cada cuadrado 3x3 se puede ver como una "celda" con coordenadas a,o en un grid 3x3 (con a,o en 1..3)

Las 9 celdas de ese cuadrado son:

Filas: $S*(a-1)+x$

Columnas: $S*(o-1)+y$

(con x,y en 1..3)

```

constraint puzzle[1,1] = 5;
constraint puzzle[1,2] = 3;
constraint puzzle[1,5] = 7;
constraint puzzle[2,1] = 6;
constraint puzzle[2,4] = 1;
constraint puzzle[2,5] = 9;
constraint puzzle[2,6] = 5;
constraint puzzle[3,2] = 9;
constraint puzzle[3,3] = 8;
constraint puzzle[3,8] = 6;
constraint puzzle[4,1] = 8;
constraint puzzle[4,5] = 6;
constraint puzzle[4,9] = 3;
constraint puzzle[5,1] = 4;
constraint puzzle[5,4] = 8;

```

```

constraint puzzle[5,6] = 3;
constraint puzzle[5,9] = 1;
constraint puzzle[6,1] = 7;
constraint puzzle[6,5] = 2;
constraint puzzle[6,9] = 6;
constraint puzzle[7,2] = 6;
constraint puzzle[7,7] = 2;
constraint puzzle[7,8] = 8;
constraint puzzle[8,4] = 4;
constraint puzzle[8,5] = 1;
constraint puzzle[8,6] = 9;
constraint puzzle[8,9] = 5;
constraint puzzle[9,5] = 8;
constraint puzzle[9,8] = 7;
constraint puzzle[9,9] = 9;

```

```

solve satisfy;

```

```

output [ "\(puzzle[i,j]) " ++

```

```

    if j == N then "\n" else "" endif | i,j in 1..N];

```

Valores iniciales

5	3			7				
6			1	9	5			
	9	8					6	
8				6				3
4			8		3			1
7				2				6
	6					2	8	
			4	1	9			5
				8			7	9

Valores finales

5 3 4 6 7 8 9 1 2
 6 7 2 1 9 5 3 4 8
 1 9 8 3 4 2 5 6 7
 8 5 9 7 6 1 4 2 3
 4 2 6 8 5 3 7 9 1
 7 1 3 9 2 4 8 5 6
 9 6 1 5 3 7 2 8 4
 2 8 7 4 1 9 6 3 5
 3 4 5 2 8 6 1 7 9

- **Para el desarrollo de la práctica, se debe utilizar el solver Gecode.**
 - Se trata de un solver completo.
 - Es decir, se tienen garantías de que, dada una codificación adecuada, este algoritmo siempre encontrará la solución o demostrará que no existe ninguna (aunque puede llevarle mucho tiempo lograrlo).
- La solución puede ser:
 - **Correcta:** la codificación no proporciona soluciones que violan alguna de las restricciones del problema.
 - **Completa:** la codificación es capaz de proporcionar todas las soluciones del problema.
- Ejemplo: proporcionar un número entero x en $[1,3]$.
 - Las soluciones $x=1$, $x=2$ y $x=3$ son correctas. Cualquier otra no.
 - La codificación $x>0 \ \&\& \ x<4$ es correcta y completa.
 - La codificación $x>0$ no es correcta (podría dar como solución $x=4$)
 - La codificación $x==1 \ || \ x==3$ no es completa (nunca proporciona la solución $x=2$)

- **Consejos generales.**

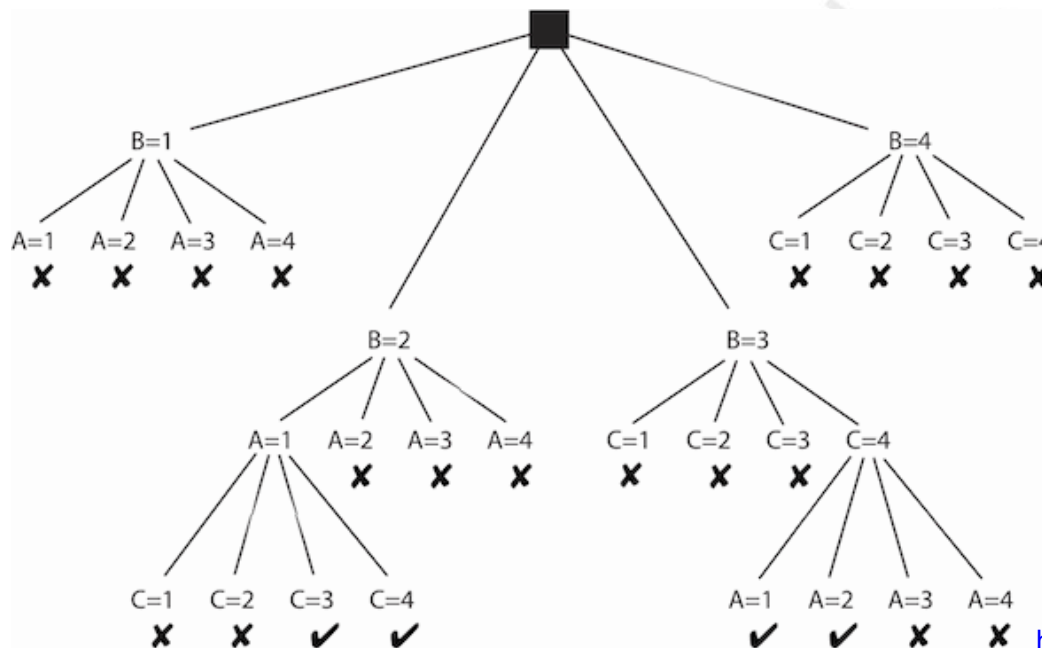
1. No hay una única forma de abordar todos los posibles problemas o ejercicios.

- Componente experimental y empírico (prueba y error), en donde tenéis que ir probando codificaciones, e iréis viendo cuáles van mejor y cuáles peor.
- A medida que vayáis resolviendo problemas, iréis adquiriendo una experiencia o intuición sobre qué funciona mejor.

• Consejos generales.

2. Subyace un proceso en el que se explora un árbol de búsqueda, y el objetivo es facilitarle el trabajo al *solver*.

- Si nuestra codificación hace que el *solver* tenga que buscar de modo excesivo se incrementarán los tiempos de resolución.
 - Es recomendable "acotar" el espacio de búsqueda y centrarse en modelar de modo adecuado la codificación de las variables
 - » Un dominio de menor tamaño implica un factor de ramificación más pequeño en el árbol, reduciendo así el espacio de búsqueda (p.ej., mejor representación entera que flotante).



- **Consejos generales.**

3. Relacionado con lo anterior, ciertas restricciones se pueden integrar en la codificación de las variables y sus dominios:

- Ej: codificar las N-reinas con 1 reina por fila (cuyo dominio es la columna de cada una) permite implícitamente eliminar la posibilidad de conflictos en cada fila, y simplifica enormemente la introducción de restricciones, como puede ser la restricción relativa a las columnas (un mero *alldifferent* sobre reinas).

4. Más ideas sobre codificación:

- MiniZinc y los *failures* de las *solving statistics*:
<https://stackoverflow.com/questions/62613164/clarification-on-failures-solver-statistic-in-minizinc>
- El número de *failures* es el número de veces que el algoritmo de búsqueda llega hasta un nodo hoja, que no es objetivo, y tiene que dar vuelta. A mayor número de *failures*, podríamos interpretar que es peor nuestra capacidad de poda, es decir, de cortar ramas y explorar otras sin necesidad de llegar a un nodo hoja. Una buena codificación puede contribuir en este sentido.

• **Consejos generales.**

5. Como pauta general:

- Usar variables con dominios finitos y ajustados ayuda a los *solvers*. No es recomendable usar “var int” sin límites. Mejor acotar rangos.
- Aunque no cambien el conjunto de soluciones, algunas restricciones redundantes pueden permitir que el *solver* pade antes.
- Pero, cuidado, en general es bueno abogar por la sencillez. Introducir demasiadas restricciones y/o variables innecesarias podría llegar a ofuscar la solución.
- Si una restricción global existe en MiniZinc y expresa exactamente lo que queremos hacer, es recomendable usarla y no reescribirla como varias restricciones pequeñas.
- Si el problema tiene soluciones equivalentes por permutación, espejo, etc., conviene añadir restricciones de *symmetry breaking* para no hacer que el *solver* explore muchas veces lo mismo con distinta apariencia. Véase siguiente diapositiva.

• Más sobre soluciones simétricas y ruptura de simetrías.

Dos soluciones son simétricas si representan esencialmente lo mismo para el problema, aunque aparezcan presentadas de forma distinta.

```
int: n = 4;
array[1..n] of var 1..n: q;
```

```
constraint all_different(q);
constraint all_different([q[i] + i | i in 1..n]);
constraint all_different([q[i] - i | i in 1..n]);
```

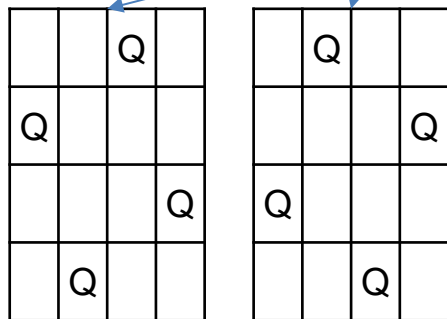
```
solve satisfy;
```

```
output ["q = \"(q)\\n\""];
```

Estructuralmente representan la misma solución. Una es el resultado de aplicar la simetría espejo a la otra: es el mismo tablero reflejado.

```
q = [3, 1, 4, 2]
```

```
-----
q = [2, 4, 1, 3]
```



```
int: n = 4;
array[1..n] of var 1..n: q;
```

```
constraint all_different(q);
constraint all_different([q[i] + i | i in 1..n]);
constraint all_different([q[i] - i | i in 1..n]);
```

```
constraint q[1] < q[n];
```

```
solve satisfy;
```

```
output ["q = \"(q)\\n\""];
```

```
q = [2, 4, 1, 3]
```

Ruptura de simetría forzando un orden.

Las soluciones simétricas hacen que el *solver* explore muchas soluciones equivalentes entre sí, ampliando innecesariamente el espacio de búsqueda y dificultando la resolución del CSP/COP.

• Un ejemplo todavía más claro...

```
include "globals.mzn";
```

```
enum DIA = {Lun, Mar, Mie};
```

```
enum ASIG = {Matematicas, Lengua, Ingles};
```

```
% Primera hora y tercera hora: una asignatura por día
```

```
array[DIA] of var ASIG: primera;
```

```
array[DIA] of var ASIG: tercera;
```

```
% Segunda hora: recreo, codificado con etiquetas 1..3
```

```
array[DIA] of var 1..3: recreo;
```

```
% En primera y tercera hora, cada asignatura
```

```
% aparece exactamente una vez
```

```
constraint all_different(primeras);
```

```
constraint all_different(terceras);
```

```
% Una asignatura no puede repetirse
```

```
% el mismo día en primera y tercera hora
```

```
constraint forall(d in DIA)(primeras[d] != terceras[d]);
```

```
% Los "recreos" se codifican con tres enteros distintos
```

```
constraint all_different(recreos);
```

```
solve satisfy;
```

Solo hay 12 horarios distintos (6 posibles asignaciones para la 1ª hora y, una vez fijada esta, solo 2 opciones válidas para la 3ª hora, ya que no se puede repetir asignatura en el mismo día).

Sin embargo, **este código devuelve 72 soluciones**, porque para cada uno de esos 12 horarios hay 6 permutaciones posibles de los enteros usados para codificar el recreo (que, aunque semánticamente representan exactamente lo mismo, para MiniZinc son valores diferentes). Estas permutaciones **son soluciones simétricas**.

```
Lun: 1ª=Matematicas, 2ª=Recreo(3), 3ª=Ingles
```

```
Mar: 1ª=Ingles, 2ª=Recreo(2), 3ª=Lengua
```

```
Mie: 1ª=Lengua, 2ª=Recreo(1), 3ª=Matematicas
```

```
-----
```

```
Lun: 1ª=Matematicas, 2ª=Recreo(2), 3ª=Ingles
```

```
Mar: 1ª=Ingles, 2ª=Recreo(3), 3ª=Lengua
```

```
Mie: 1ª=Lengua, 2ª=Recreo(1), 3ª=Matematicas
```

```
-----
```

```
Lun: 1ª=Matematicas, 2ª=Recreo(3), 3ª=Ingles
```

```
Mar: 1ª=Ingles, 2ª=Recreo(1), 3ª=Lengua
```

```
Mie: 1ª=Lengua, 2ª=Recreo(2), 3ª=Matematicas
```

```
-----
```

```
...
```

- **Consejos generales.**

6. Se aconseja revisar la documentación de MiniZinc sobre prácticas de modelado recomendadas:

<https://docs.minizinc.dev/en/stable/efficient.html>

2.7. Effective Modelling Practices in MiniZinc

2.7.1. Variable Bounds

2.7.2. Effective Generators

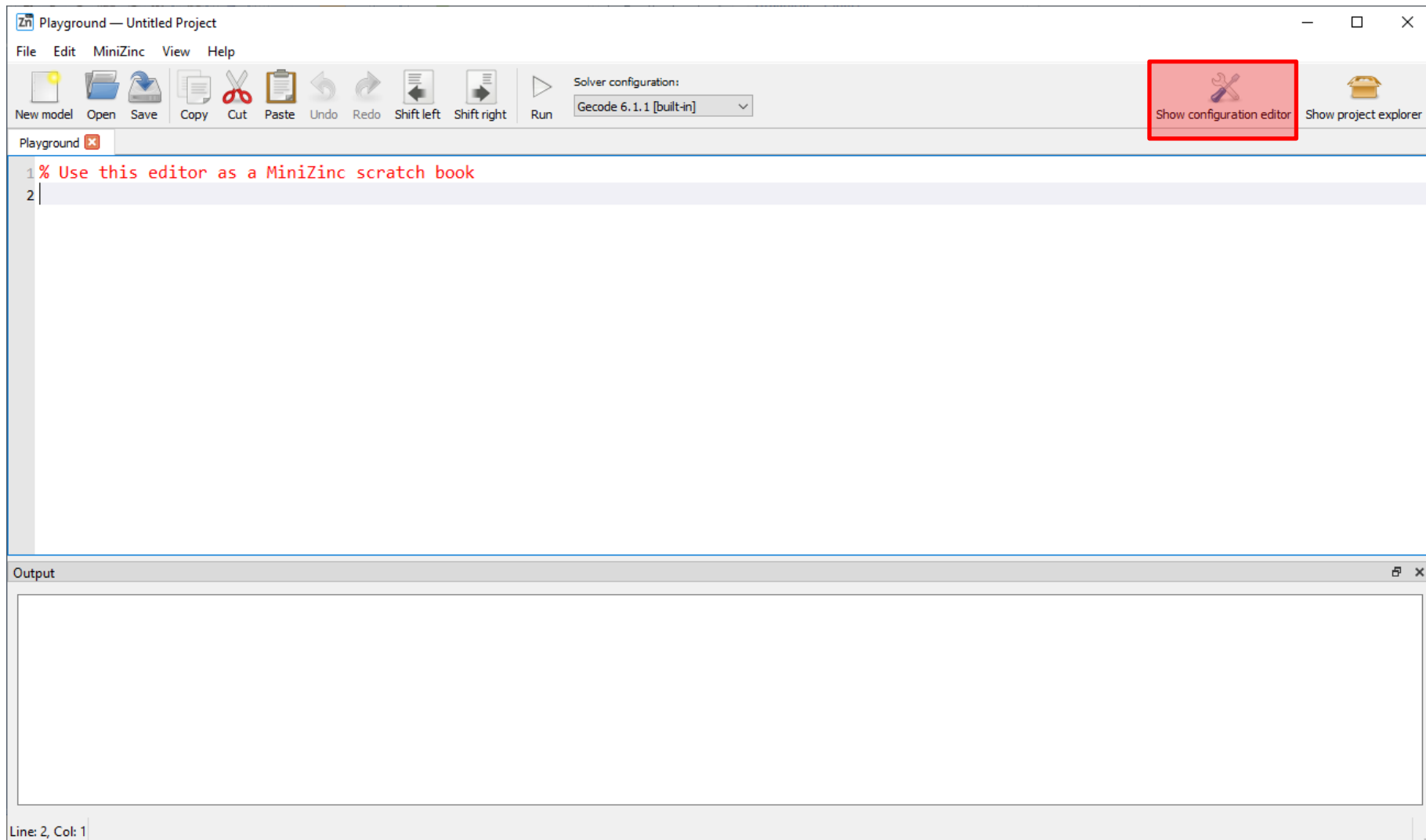
2.7.3. Redundant Constraints

2.7.4. Modelling Choices

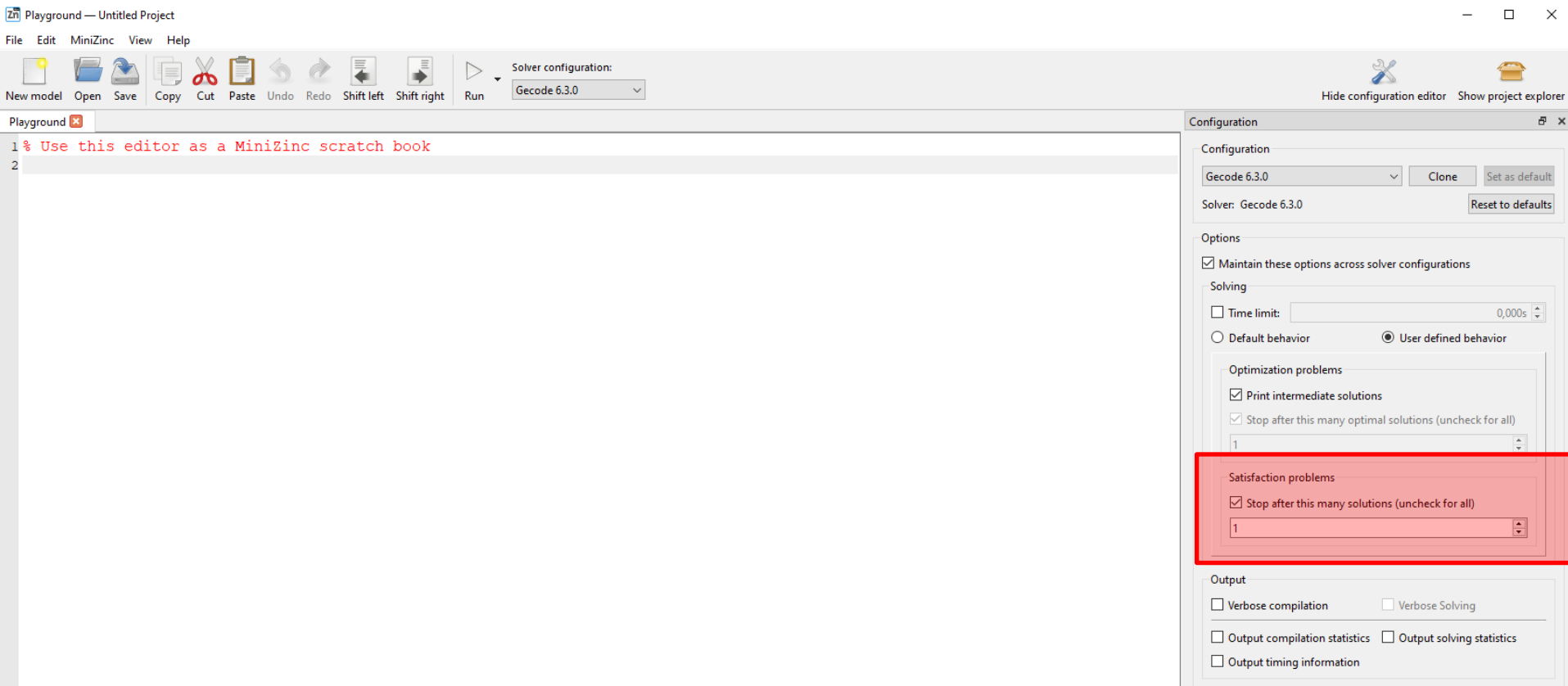
2.7.5. Multiple Modelling and Channels

⊕ 2.7.6. Symmetry

- MiniZinc permite **imprimir todas las soluciones de una codificación**. Esto permite **verificar que una codificación es correcta y completa**.

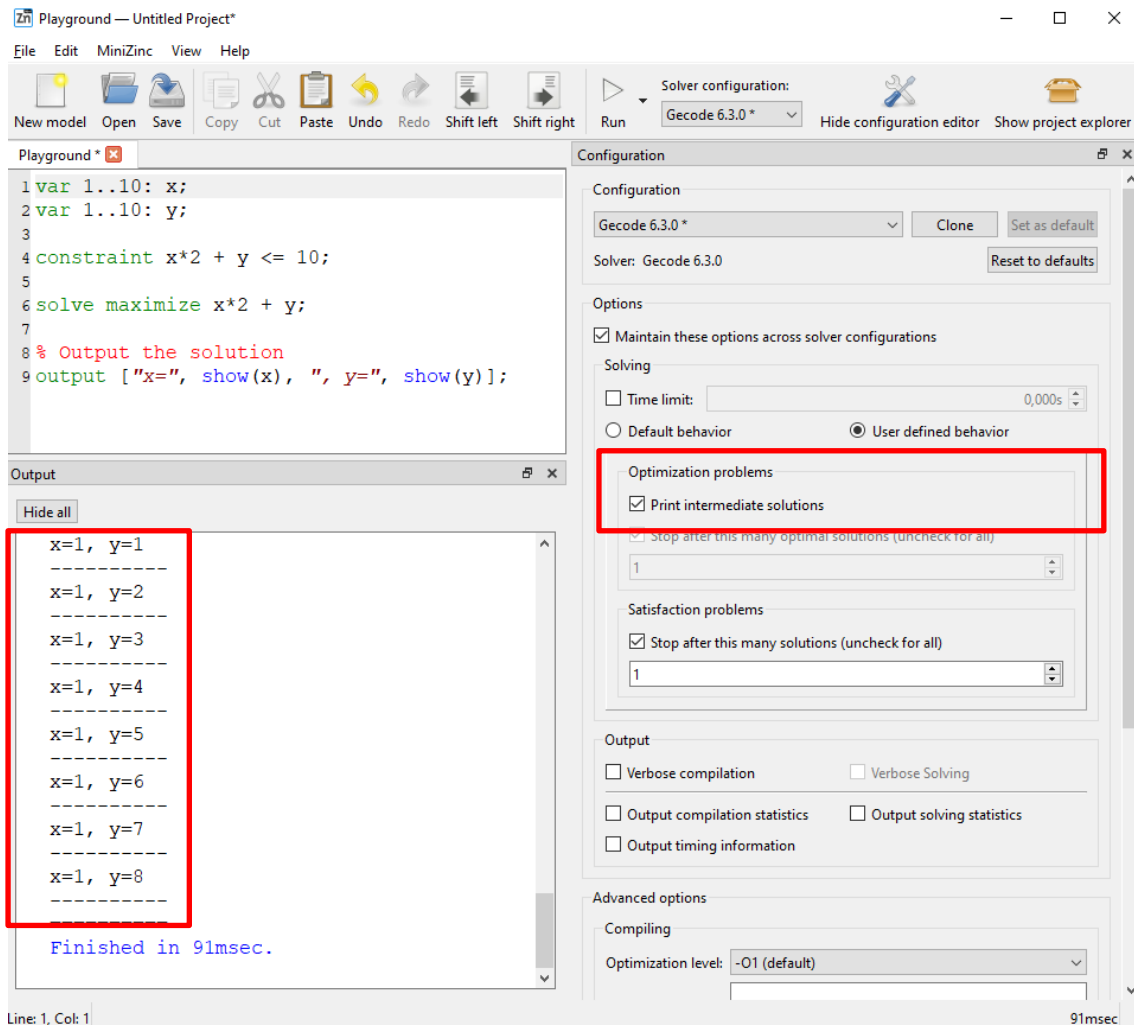


- "Show configuration editor" - "User-defined behavior" – "Satisfaction problems" - "Stop after this many solutions (uncheck for all)".



The screenshot shows the MiniZinc Playground interface. The main editor area contains the text: `1 % Use this editor as a MiniZinc scratch book` and `2`. The right-hand panel displays the "Configuration" dialog box. In this dialog, the "Options" section is expanded, and the "User defined behavior" radio button is selected. Under the "Solving" section, the "Optimization problems" group is expanded, and the "Satisfaction problems" group is also expanded. The checkbox "Stop after this many solutions (uncheck for all)" is checked, and the value "1" is entered in the adjacent spin box. The "Output" section at the bottom is also visible, with several checkboxes for verbose compilation, solving, and statistics.

- Cuando MiniZinc se enfrenta a un **problema de optimización** (COP) y se marca la opción "Print intermediate solutions", lo que hace es mostrar **todas las etapas intermedias que han llevado al solver hasta el óptimo**.



The screenshot shows the MiniZinc Playground interface. The main editor contains the following code:

```
1 var 1..10: x;
2 var 1..10: y;
3
4 constraint x*2 + y <= 10;
5
6 solve maximize x*2 + y;
7
8 % Output the solution
9 output ["x=", show(x), ", y=", show(y)];
```

The Output window shows the following intermediate solutions:

```
x=1, y=1
-----
x=1, y=2
-----
x=1, y=3
-----
x=1, y=4
-----
x=1, y=5
-----
x=1, y=6
-----
x=1, y=7
-----
x=1, y=8
-----
```

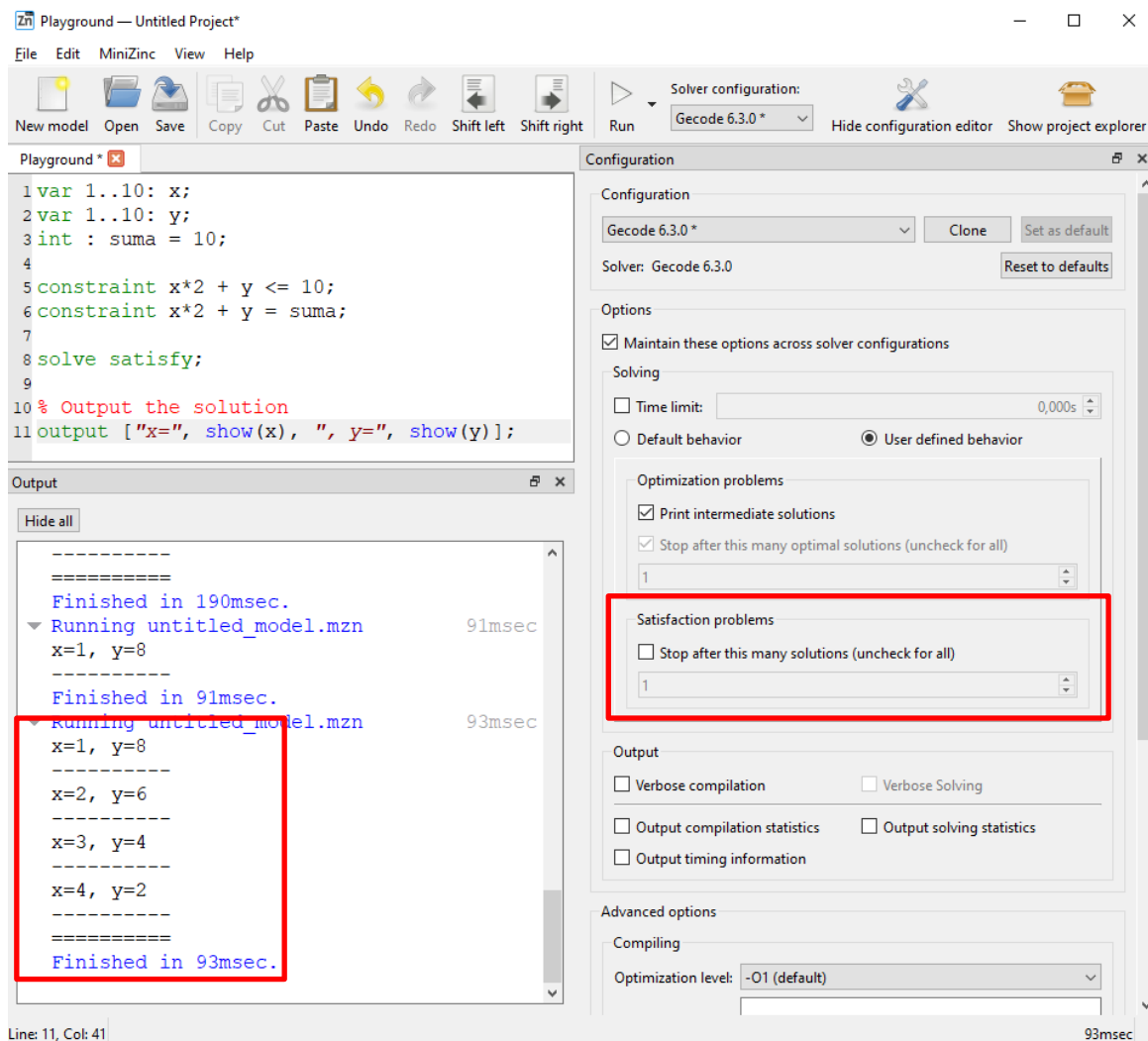
The Configuration window on the right shows the following settings:

- Solver configuration: Gecode 6.3.0 *
- Solver: Gecode 6.3.0
- Options:
 - ☒ Maintain these options across solver configurations
 - Solving:
 - ☐ Time limit: 0,000s
 - ☐ Default behavior
 - ☒ User defined behavior
 - Optimization problems:
 - ☒ Print intermediate solutions
 - Satisfaction problems:
 - ☒ Stop after this many solutions (uncheck for all): 1
- Output:
 - ☐ Verbose compilation
 - ☐ Verbose Solving
 - ☐ Output compilation statistics
 - ☐ Output solving statistics
 - ☐ Output timing information
- Advanced options:
 - Compiling:
 - Optimization level: -O1 (default)

The status bar at the bottom indicates "Finished in 91msec."

- **MiniZinc, por defecto, te devuelve la primera solución óptima que encuentra.**

Si queremos resolver un COP y buscamos todas las soluciones óptimas (asumiendo que hay varios óptimos locales, es decir, configuraciones que proporcionan el mismo valor óptimo de la función a minimizar), **podríamos ejecutar el COP, guardar el valor óptimo obtenido, y luego ejecutar un CSP (*solve satisfy;*) con una restricción que fuerce a que el coste sea ese.**



The screenshot shows the MiniZinc Playground interface. The code editor contains the following MiniZinc code:

```
1 var 1..10: x;
2 var 1..10: y;
3 int : suma = 10;
4
5 constraint x*2 + y <= 10;
6 constraint x*2 + y = suma;
7
8 solve satisfy;
9
10 % Output the solution
11 output ["x=", show(x), ", y=", show(y)];
```

The Output panel shows the results of solving the model:

```
-----
=====
Finished in 190msec.
Running untitled_model.mzn 91msec
x=1, y=8
-----
Finished in 91msec.
Running untitled_model.mzn 93msec
x=1, y=8
-----
x=2, y=6
-----
x=3, y=4
-----
x=4, y=2
-----
Finished in 93msec.
```

The Configuration panel on the right shows the solver settings:

- Solver configuration: Gecode 6.3.0 *
- Solver: Gecode 6.3.0
- Options:
 - ☒ Maintain these options across solver configurations
 - Solving:
 - ☐ Time limit: 0,000s
 - ☐ Default behavior
 - ☒ User defined behavior
 - Optimization problems:
 - ☒ Print intermediate solutions
 - ☒ Stop after this many optimal solutions (unchecked for all)
 - Satisfaction problems (highlighted with a red box):
 - ☐ Stop after this many solutions (unchecked for all)
- Output:
 - ☐ Verbose compilation
 - ☐ Verbose Solving
 - ☐ Output compilation statistics
 - ☐ Output solving statistics
 - ☐ Output timing information
- Advanced options:
 - Compiling:
 - Optimization level: -O1 (default)

- La práctica consiste en resolver 7 problemas de satisfacción/optimización de restricciones (ver enunciado).
- Se entregará una memoria donde se desarrollará la respuesta razonada a las preguntas planteadas en cada problema, se justificará la codificación empleada, y se presentará razonadamente el uso que se haya hecho de herramientas de IA generativa.
- Cada problema tiene una puntuación que oscila entre 1 y 2 puntos, contemplando tanto la codificación, la cual debe ser **correcta y completa** (las soluciones que da son correctas y no da ninguna solución incorrecta), como la respuesta/explicación de la memoria.
- Al igual que en la P1, se contempla la posibilidad de realizar **defensa oral** de un porcentaje de las entregas realizadas.
- La entrega consistirá en un **fichero ZIP** que contenga los 7 ficheros **MZN** correspondientes al **código MiniZinc** de cada ejercicio y el **PDF** de la memoria.
- Los ejercicios que tengan **errores sintácticos en MiniZinc automáticamente califican con 0 (cero) puntos**.

Fecha de entrega: 12 de mayo de 2026 23:59



UNIVERSIDAD
DE GRANADA



Técnicas de los Sistemas Inteligentes

Grado en Ingeniería Informática

Curso 2025-26. Seminario 2 Razonamiento con Restricciones

Pablo Mesejo Santiago y David Criado Ramón

Departamento de Ciencias de la
Computación e Inteligencia Artificial
<http://decsai.ugr.es>